



杭州电子科技大学

实验一 开关量输入输出实验

课 程 微机原理与接口技术

学 院 自动化学院

专 业 自动化 (新工科试验班)

学 号 24061428

学生姓名 张跃哲

指导教师 金朝阳

完成日期 2026 年 4 月 17 日

目录

1	实验要求与实验目的	1
1.1	实验要求	1
1.2	实验目的	1
1.3	设计提示	1
2	Proteus 原理图与硬件设计说明	1
2.1	原理图截图	2
2.2	电路组成与功能对应关系	2
2.3	主要元件清单	3
3	软件设计与流程分析	3
3.1	程序流程图	4
3.2	完整实验代码	4
3.3	关键代码逐段说明	5
4	调试过程与实验结果	5
4.1	调试过程记录	5
4.2	结果验证表	6
4.3	编译信息	6
5	问题分析与改进建议	6
6	实验结论	6
A	附录：编译清单摘要	6

1 实验要求与实验目的

1.1 实验要求

根据实验指导书，实验一的要求如下：

1. 利用 AT89C51 微控制器的 P0 口作开关量输入口，P1 口作开关量输出口。
2. 当 P0.x 端开关闭合时，对应 P1.x 口的 LED 发光二极管点亮；当 P0.x 端开关断开时，对应 P1.x 口的 LED 发光二极管熄灭。
3. 画出 AT89C51 实现上述功能的完整电路图，包括微控制器电源、复位电路、晶振电路和控制电路。
4. 完成全部程序和电路调试工作。

1.2 实验目的

1. 掌握 AT89C51 微控制器最基本电路的设计方法。
2. 了解微控制器 I/O 端口的使用方法。

1.3 设计提示

P0 口作为 I/O 使用时，需要配置上拉电阻，避免端口悬空导致读值不稳定。

2 Proteus 原理图与硬件设计说明

2.1 原理图截图

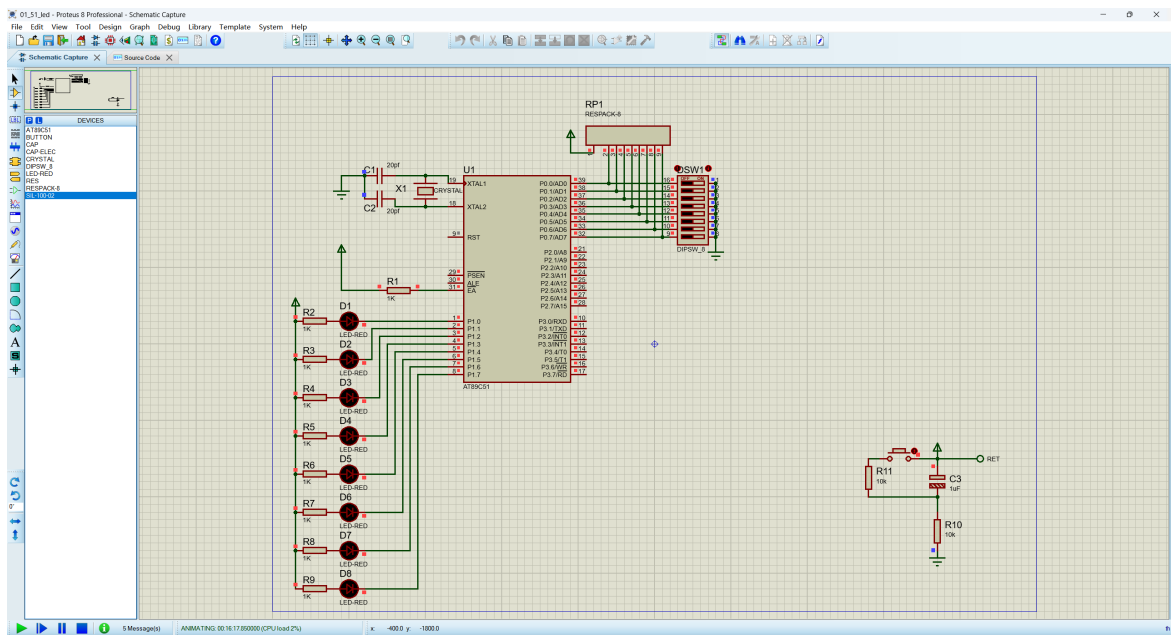


图 1 实验一 Proteus 原理图截图

2.2 电路组成与功能对应关系

本实验硬件电路由四个部分组成：

- 电源与基础引脚：为 AT89C51 提供稳定工作电源，并完成必要控制引脚配置。
- 复位电路：保证上电时单片机进入确定初始状态，程序从复位入口开始执行。
- 晶振电路：由晶振和电容构成时钟网络，为 CPU 提供系统时钟。
- I/O 控制电路：P0 连接拨码开关输入，P1 连接 LED 输出，完成开关状态到灯状态的位对应映射。

结合原理图可见：

1. SW1（DIPSW_8）提供 8 位开关量输入；
2. P0.0-P0.7 接收上述输入信号；
3. P1.0-P1.7 分别驱动 D1-D8；
4. 每个 LED 串联电阻，起限流与保护作用。

2.3 主要元件清单

表 1 实验主要元件与模型

序号	元件名称	元件规格	所在元件库	工具模型
1	微控制器	AT89C51	Microprocessor	Component mode
2	按钮/开关	BUTTON	Switches & Relay	Component mode
3	晶振	CRYSTAL	Miscellaneous	Component mode
4	发光二极管	LED-RED	Optoelectronics	Component mode
5	电容	CAP / CAP-ELEC	Capacitors	Component mode
6	电阻	RES	Resistors	Component mode
7	拨码开关	DIPSW_8	Switches & Relay	Component mode
8	电源与地	POWER / GROUND	Terminals	Terminals mode
9	电源输入端	SIL-100-02	Connectors	Component mode

3 软件设计与流程分析

3.1 程序流程图

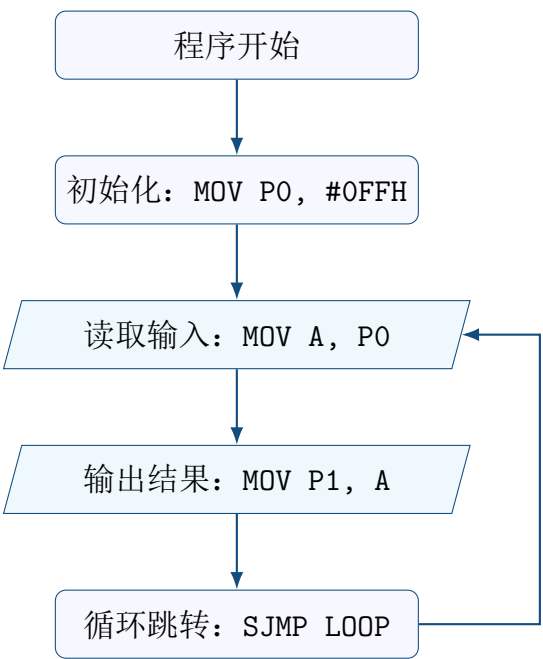


图 2 实验一程序流程图

3.2 完整实验代码

Listing 1 01_single_led.A51 完整源码

```
1  ORG 0000H
2  LJMP MAIN
3
4  ORG 0030H
5  MAIN:
6  LOOP:
7      MOV P0, #0FFH
8      MOV A, P0
9      MOV P1, A
10     SJMP LOOP
11  END
```

3.3 关键代码逐段说明

表 2 关键指令功能说明

指令或语句	作用说明
ORG 0000H + LJMP MAIN	定义复位入口，单片机上电后从 0000H 跳转到主程序地址。
ORG 0030H	主程序起始地址定义，便于与中断向量区域区分。
MOV P0, #0FFH	将 P0 各位写 1，使端口处于读取外部开关状态的工作方式。
MOV A, P0	将 P0 当前 8 位输入状态读入累加器 A。
MOV P1, A	将输入值直接输出到 P1，实现 8 位一一对应控制。
SJMP LOOP	构成无限循环，实现实时轮询与实时响应。

4 调试过程与实验结果

4.1 调试过程记录

1. 在 Keil 中编译 01_single_led.A51，确认语法与目标文件生成正常。
2. 在 Proteus 中连接 AT89C51、拨码开关、LED、晶振、复位和供电电路。
3. 加载编译生成程序，启动仿真。
4. 逐位拨动 SW1，观察 D1–D8 是否按位同步变化。
5. 对全灭、全亮、奇偶位组合等典型输入进行对照验证。

4.2 结果验证表

表 3 输入输出对应关系验证

测试序号	P0 输入（开关状态）	预期 P1/LED 输出	结果
1	0x00	全灭	符合
2	0xFF	全亮	符合
3	0x55	奇数位亮、偶数位灭	符合
4	0xAA	偶数位亮、奇数位灭	符合
5	任意组合	位对位同步显示	符合

4.3 编译信息

从 A51 编译列表可确认：

- 汇编器版本：A51 V8.2.7.0；
- 关键符号地址：MAIN=0030H，LOOP=0030H，P0=0080H，P1=0090H；
- 编译结果：0 warning，0 error。

5 问题分析与改进建议

1. P0 口使用时应确认上拉网络有效，避免输入悬空导致 LED 状态抖动。
2. 实际硬件中可加入按键消抖或软件延时，提高状态显示稳定性。
3. 可扩展为中断方式采样，减少无效轮询占用。

6 实验结论

本实验完成了基于 AT89C51 的开关量输入输出控制：P0 口读取开关状态，P1 口输出 LED 控制信号，最终实现输入与输出一一对应。通过本次实验，验证了单片机最基本电路（电源、复位、晶振、I/O 控制）与端口编程方法，达到实验指导书要求。

A 附录：编译清单摘要

Listing 2 01_single_led.LST（节选）

```
1 A51 MACRO ASSEMBLER 01_SINGLE_LED
04/17/2026
01:21:37 PAGE 1
2
3
4 MACRO ASSEMBLER A51 V8.2.7.0
5 OBJECT MODULE PLACED IN 01_single_led.OBJ
6 ASSEMBLER INVOKED BY: C:\Keil\C51\BIN\A51.exe E:\homework\51_proteus
  \01_single_led\01_single_led.A51 DEBUG SYMBOLS OBJEC
7 T(01_single_led.OBJ)
8
9 LOC OBJ LINE SOURCE
10
11 0000 1 ORG 0000H
12 0000 020030 2 LJMP MAIN
13 3
14 0030 4 ORG 0030H
15 0030 5 MAIN:
16 0030 6 LOOP:
17 0030 7580FF 7 MOV P0, #0FFH
18 0033 E580 8 MOV A, P0
19 0035 F590 9 MOV P1, A
20 0037 80F7 10 SJMP LOOP
21 11 END

22 A51 MACRO ASSEMBLER 01_SINGLE_LED
04/17/2026
01:21:37 PAGE 2
23
24 SYMBOL TABLE LISTING
25 -----
26
27
28 N A M E T Y P E V A L U E A T T R I B U T E S
29
30 LOOP . . . . . C ADDR 0030H A
31 MAIN . . . . . C ADDR 0030H A
32 P0 . . . . . D ADDR 0080H A
33 P1 . . . . . D ADDR 0090H A
34
```

```
35  
36 REGISTER BANK(S) USED: 0  
37  
38  
39 ASSEMBLY COMPLETE. 0 WARNING(S), 0 ERROR(S)
```



杭州电子科技大学

实验二 LED 轮换点亮实验

课 程 微机原理与接口技术

学 院 自动化学院

专 业 自动化 (新工科试验班)

学 号 24061428

学生姓名 张跃哲

指导教师 金朝阳

完成日期 2026 年 4 月 17 日

目录

1	实验要求与实验目的	1
1.1	实验要求	1
1.2	实验目的	1
1.3	设计提示	1
2	Proteus 原理图与硬件电路说明	1
2.1	完整原理图	2
2.2	分电路截图	2
2.3	硬件工作原理	4
2.4	主要元件清单	4
3	软件设计与程序流程	4
3.1	设计思路	4
3.2	程序流程图	5
3.3	完整实验代码	5
3.4	关键代码说明	6
4	调试过程与实验结果	6
4.1	调试过程记录	6
4.2	结果验证表	7
5	问题分析与改进建议	7
6	实验结论	7

1 实验要求与实验目的

1.1 实验要求

根据实验指导书，实验二（LED 轮换点亮实验）的要求如下：

1. 利用 AT89C51 单片机的 P1 口进行扩展，驱动 LED 灯控制输出。
2. 编写程序，使 P1.0~P1.7 上的发光二极管循环点亮。
3. 画出 STC89C51/AT89C51 实现上述功能的完整电路图，包括单片机电源、复位电路、晶振电路和控制电路。
4. 完成全部程序和电路调试工作。

1.2 实验目的

1. 掌握 AT89C51 单片机并行输出口作为 I/O 输出电路的设计方法。
2. 学习循环移位指令在流水灯控制中的应用。

1.3 设计提示

1. 程序设计采用软件延时方法控制灯的轮换速度。
2. 可采用循环左移指令实现从 P1.0 到 P1.7 的轮换点亮。

2 Proteus 原理图与硬件电路说明

2.1 完整原理图

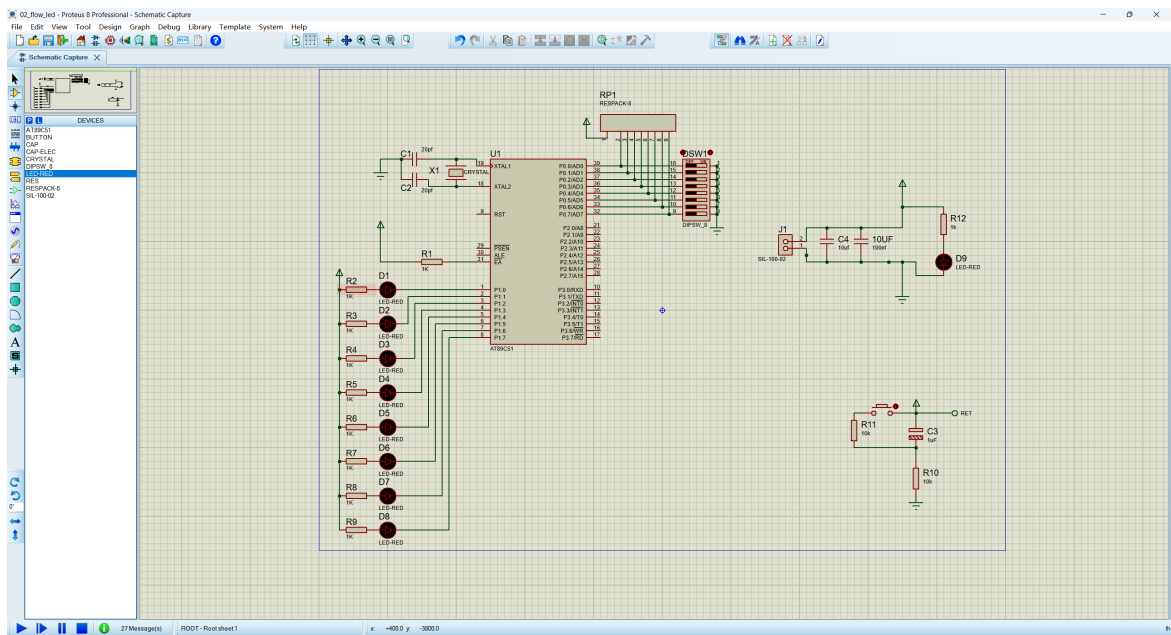


图 1 实验二完整原理图

2.2 分电路截图

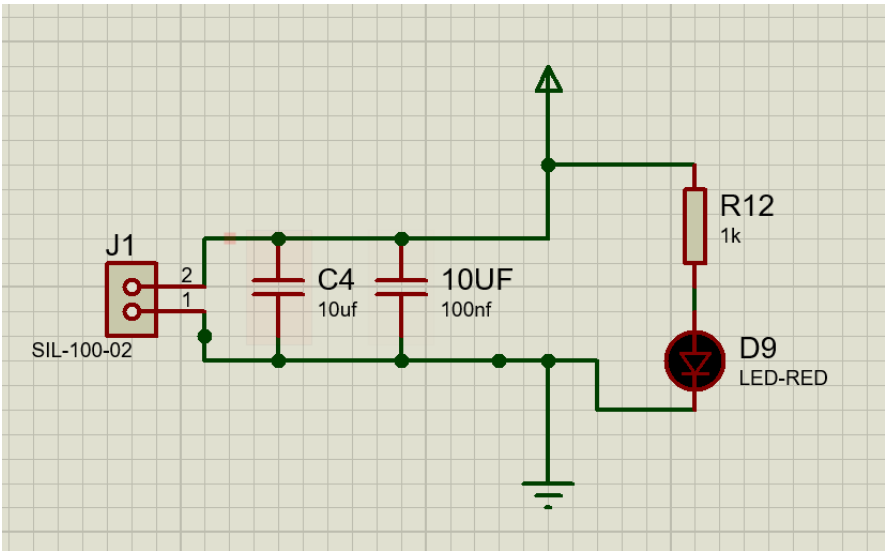


图 2 电源电路

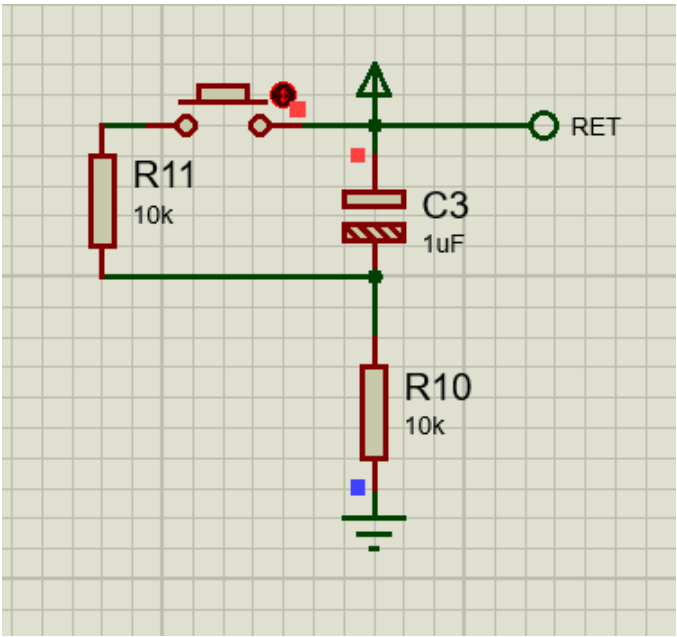


图 3 复位电路

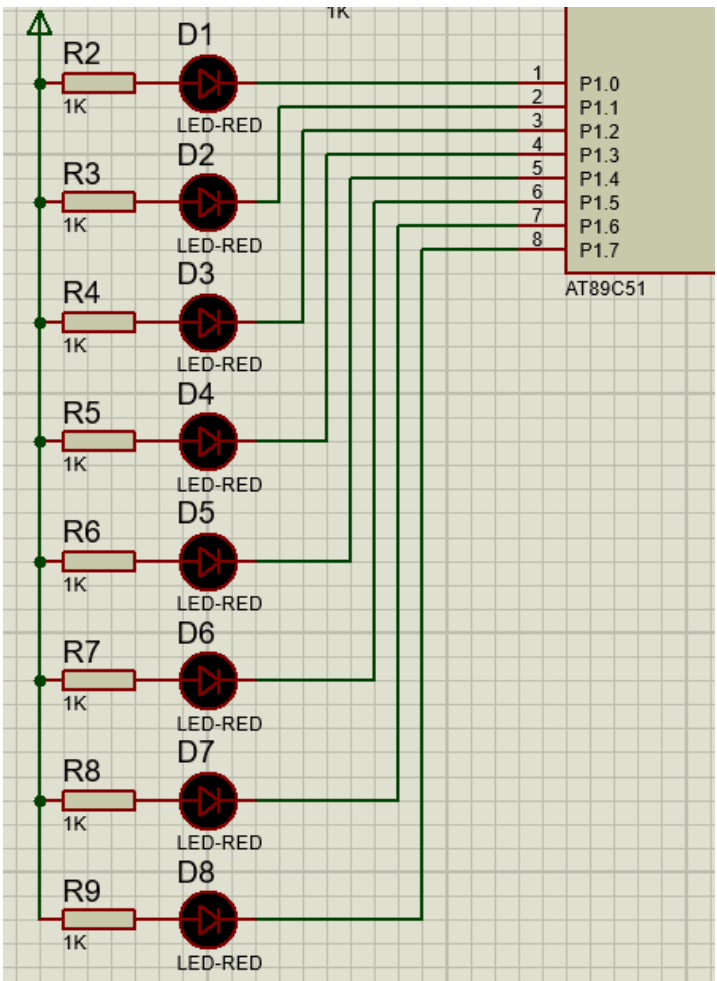


图 4 LED 控制电路（P1.0~P1.7）

2.3 硬件工作原理

本实验以 AT89C51 为核心，硬件由以下四部分构成：

- 电源电路：提供 +5V 稳定供电，并使用电容进行滤波与去耦。
- 复位电路：上电后为单片机提供可靠复位脉冲，保证程序从入口地址开始执行。
- 晶振电路：由晶振与电容构成时钟网络，提供 MCU 运行时钟。
- LED 控制电路：P1.0~P1.7 分别连接 D1~D8，实现按位输出控制。

结合电路连接，程序通过改变 P1 端口输出位模式，实现 8 个 LED 的依次轮换点亮效果。

2.4 主要元件清单

表 1 实验二主要元件与模型

序号	元件名称	元件规格	所在元件库	所在工具模型
1	单片机	AT89C51	Microprocessor	Component mode
2	按钮	BUTTON	Switchs & Relay	Component mode
3	晶振	CRYSTAL	Miscellaneous	Component mode
4	发光二极管	LED-RED	Optoelectronics	Component mode
5	电容	CAP	Capacitors	Component mode
6	电解电容	CAP-ELEC	Capacitors	Component mode
7	电阻	RES	Resistors	Component mode
9	电源	POWER	Terminals	Terminals mode
10	地	GROUND	Terminals	Terminals mode
11	电源输入端	SIL-100-02	connectors	Component mode

3 软件设计与程序流程

3.1 设计思路

为了实现 LED 轮换点亮，程序采用“单 0 位循环左移 + 软件延时”的方法：

1. 将累加器初始化为 0FEH，即仅最低位为 0；

- 2. 将累加器值输出到 P1，点亮当前目标 LED；
- 3. 调用延时子程序控制切换节奏；
- 4. 使用 RL A 循环左移，使“0 位”在 8 位端口中循环移动；
- 5. 进入无限循环，得到连续流水灯效果。

3.2 程序流程图

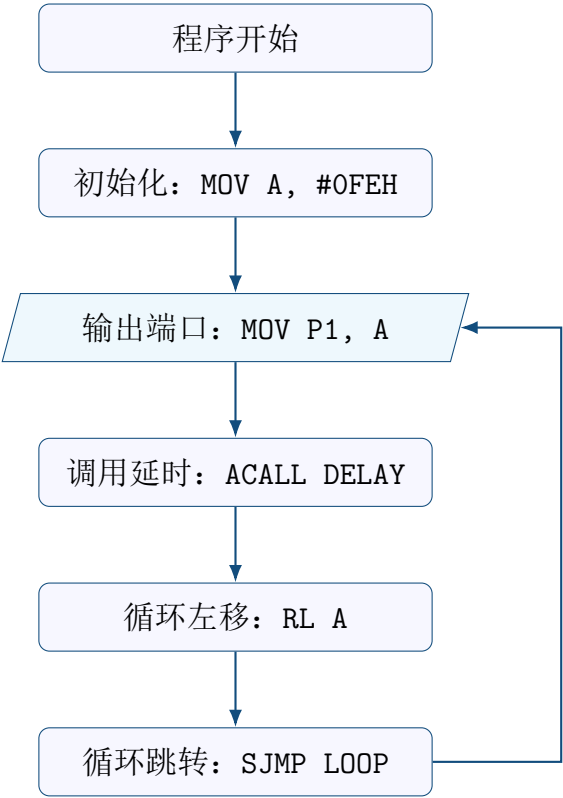


图 5 LED 轮换点亮程序流程图

3.3 完整实验代码

Listing 1 02_flow_led.A51 完整源码

```
1  ORG 0000H
2  LJMP MAIN
3
4  ORG 0030H
5  MAIN:
6      MOV A, #0FEH          ; active-low start at P1.0
7  LOOP:
8      MOV P1, A             ; output one-hot low pattern
```

```

9      ACALL DELAY
10     RL A                      ; circular left shift
11     SJMP LOOP
12
13 DELAY:
14     MOV R7, #200
15 D1:
16     MOV R6, #250
17 D2:
18     DJNZ R6, D2
19     DJNZ R7, D1
20     RET
21
22 END
```

3.4 关键代码说明

表 2 关键指令与功能说明

指令	功能说明
MOV A, #0FEH	初始化输出模式，保证仅一个 LED 处于点亮位。
MOV P1, A	将模式输出到 P1 口，驱动 D1~D8 中当前目标 LED。
ACALL DELAY	调用软件延时子程序，控制视觉上的切换速度。
RL A	将累加器按位循环左移，实现点亮位置依次向高位移动。
DJNZ R6, D2 / DJNZ R7, D1	双层计数延时，提供稳定时间基准。
SJMP LOOP	构成主循环，持续输出流水灯效果。

4 调试过程与实验结果

4.1 调试过程记录

- 1. 根据原理图完成电源、复位、晶振和 LED 控制电路连接。
- 2. 加载汇编程序并启动 Proteus 仿真。
- 3. 观察 D1~D8 是否按顺序轮换点亮。

- 4. 调整延时常数，确认轮换速度满足观察要求。
- 5. 连续运行，检查是否存在跳灯、乱序或停滞现象。

4.2 结果验证表

表 3 LED 轮换点亮结果记录

测试步骤	预期现象	结果
1	D1 点亮，其余熄灭	符合
2	D2 点亮，其余熄灭	符合
3	D3 点亮，其余熄灭	符合
4	D4 点亮，其余熄灭	符合
5	D5 点亮，其余熄灭	符合
6	D6 点亮，其余熄灭	符合
7	D7 点亮，其余熄灭	符合
8	D8 点亮后回到 D1，循环往复	符合

5 问题分析与改进建议

- 1. 若 LED 切换过快，可增大延时循环计数值提高可视性。
- 2. 若出现点亮逻辑相反，应检查 LED 接线极性与端口灌电流方式。
- 3. 后续可引入定时器中断替代软件延时，以提高程序可扩展性。

6 实验结论

本实验完成了基于 AT89C51 的 LED 轮换点亮控制。程序通过 P1 口并行输出、循环左移指令与软件延时子程序，实现了 D1~D8 依次点亮并循环运行。实验验证了并行输出口应用与循环移位指令在单片机控制中的典型用法，达到了实验二要求。

篤學力行
守正求新



杭州电子科技大学

实验三 交通灯控制实验

课 程 微机原理与接口技术

学 院 自动化学院

专 业 自动化 (新工科试验班)

学 号 24061428

学生姓名 张跃哲

指导教师 金朝阳

完成日期 2026 年 4 月 27 日

目录

1	实验要求与实验目的	1
1.1	实验要求	1
1.2	实验目的	1
1.3	总体设计思路	1
2	Proteus 原理图与硬件电路说明	1
2.1	完整原理图	2
2.2	分电路截图	2
2.3	P1 口灯位分配	5
2.4	主要元件清单	5
3	软件设计与程序流程	5
3.1	交通灯状态表	6
3.2	程序流程图	7
3.3	INT0 应急中断流程	8
3.4	交通灯状态时序	8
3.5	完整实验代码	8
3.6	关键代码说明	13
4	调试过程与实验结果	13
4.1	调试过程记录	13
4.2	结果验证表	14
4.3	程序正确性分析	14
5	问题分析与改进建议	14
6	实验结论	14

1 实验要求与实验目的

1.1 实验要求

本实验围绕 AT89C51 单片机完成十字路口交通灯控制系统设计，要求如下：

1. 使用 P1 口驱动东西、南北两个方向的红、黄、绿 LED 信号灯。
2. 按交通放行顺序实现绿灯通行、绿灯闪烁提示、黄灯过渡和红灯等待。
3. 使用 INT0 外部中断接入应急按钮，触发后进入全红状态，延时后恢复原交通状态。
4. 在 Proteus 中完成电源、复位、晶振、灯组控制和外部中断电路连接，并完成仿真调试。

1.2 实验目的

1. 掌握 AT89C51 并行 I/O 口驱动多路 LED 负载的方法。
2. 理解交通灯有限状态机的设计思路和状态切换关系。
3. 学习 8051 外部中断 INT0 的初始化、入口设置、现场保护和中断返回。
4. 提高 Proteus 仿真环境下软硬件联合调试能力。

1.3 总体设计思路

系统以 P1 口输出六路交通灯控制信号，以 P3.2 作为 INT0 应急输入。正常运行时，程序按照“南北绿灯通行、南北黄灯过渡、东西绿灯通行、东西黄灯过渡”的顺序循环；当 INT0 按键触发时，程序保存当前 P1 状态，临时切换到东西、南北全红状态，延时结束后恢复中断前的灯态。

2 Proteus 原理图与硬件电路说明

2.1 完整原理图

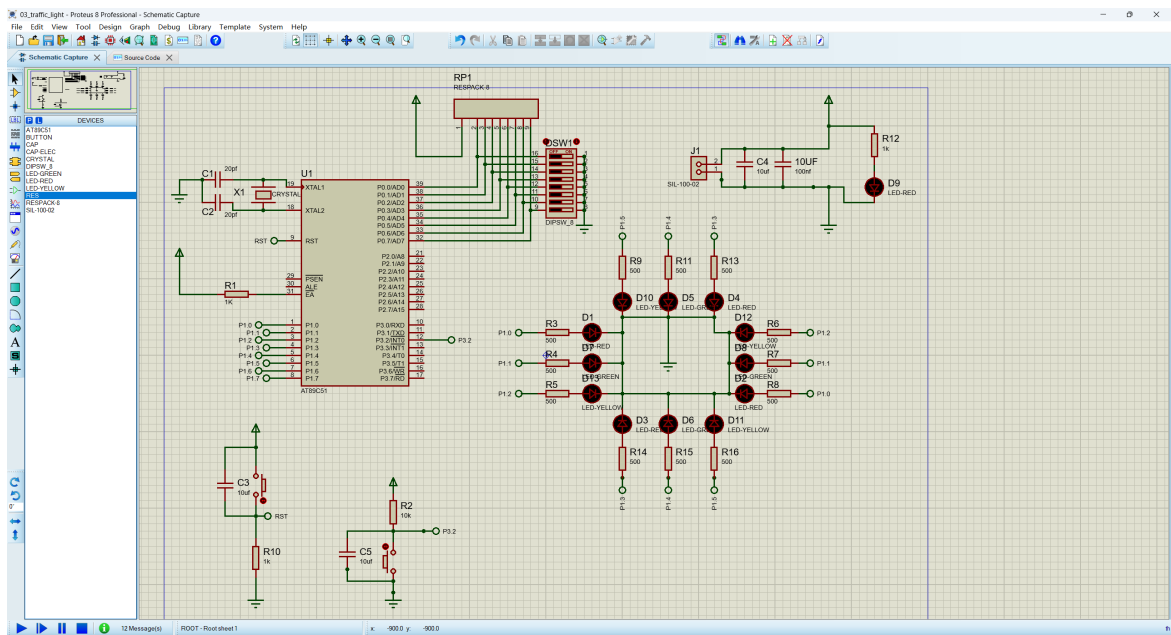


图 1 实验三交通灯 Proteus 完整原理图

2.2 分电路截图

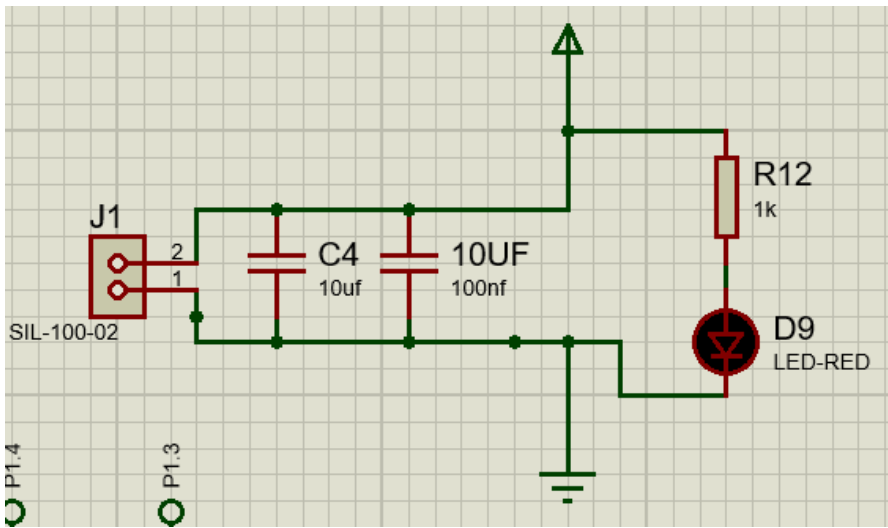


图 2 单片机供电电路

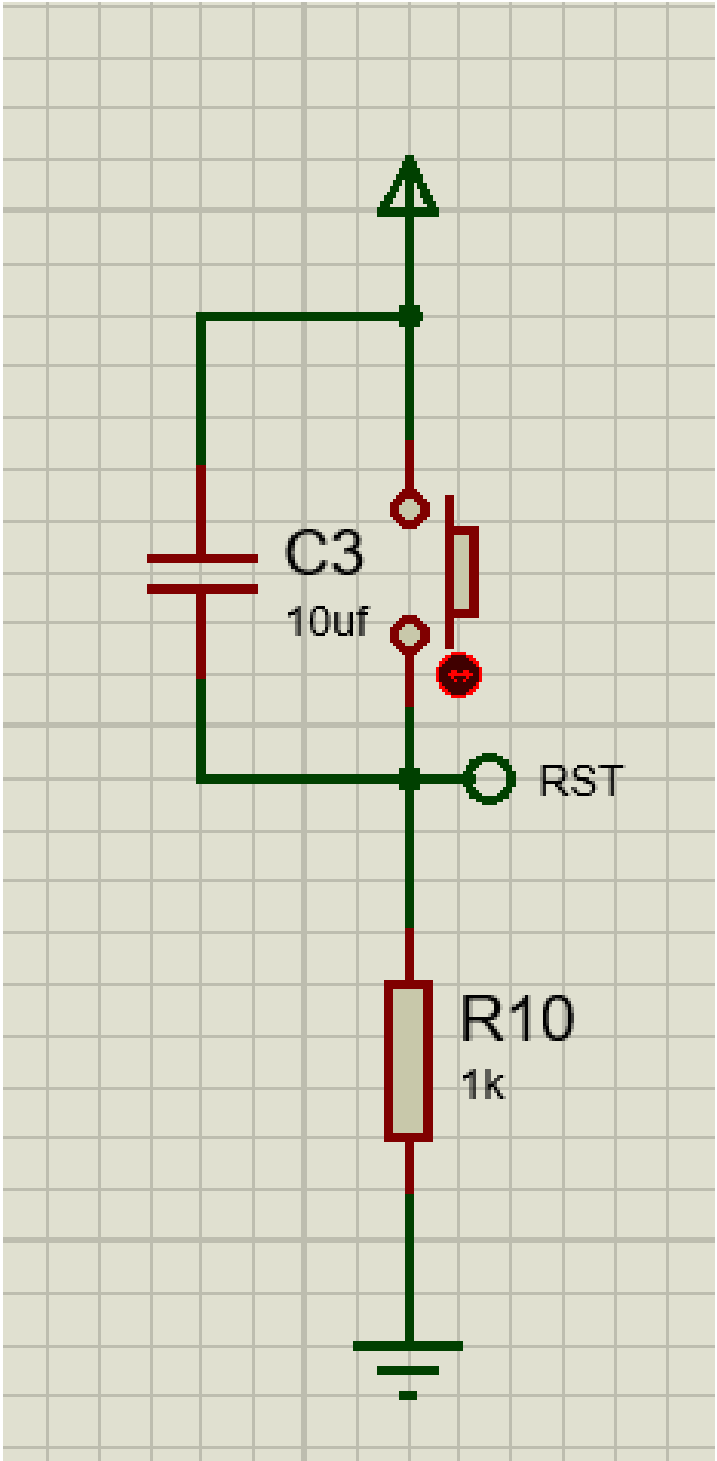


图 3 复位电路

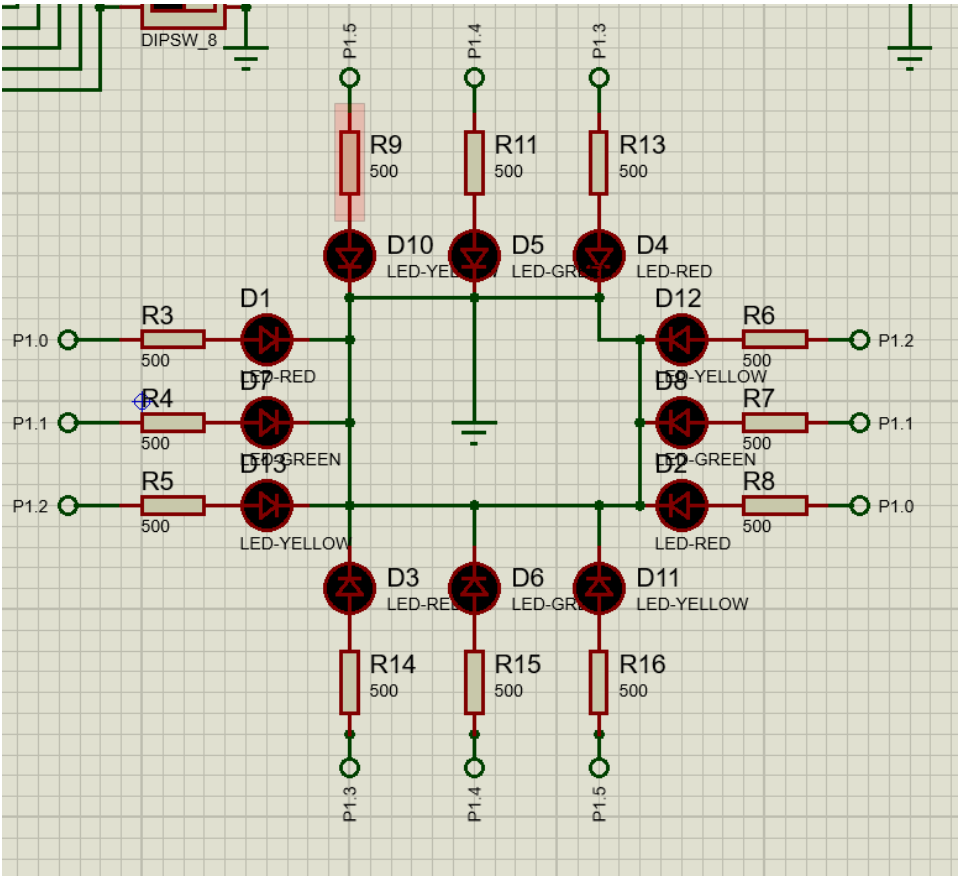


图 4 交通灯 LED 控制电路

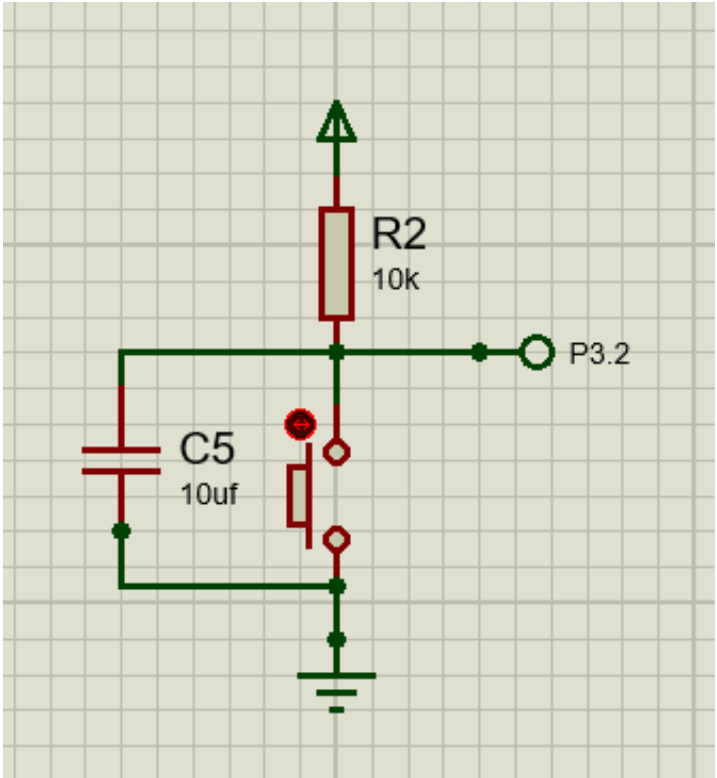


图 5 INT0 应急按键输入电路

2.3 P1 口灯位分配

表 1 P1 口交通灯控制位定义

端口位	控制对象	点亮电平	说明
P1.0	东西红灯	高电平	东西方向禁止通行
P1.1	东西绿灯	高电平	东西方向允许通行
P1.2	东西黄灯	高电平	东西方向通行结束提示
P1.3	南北红灯	高电平	南北方向禁止通行
P1.4	南北绿灯	高电平	南北方向允许通行
P1.5	南北黄灯	高电平	南北方向通行结束提示

2.4 主要元件清单

表 2 实验三主要元件与模型

序号	元件名称	元件规格	功能说明
1	单片机	AT89C51	主控制器，输出交通灯状态并响应 INT0
2	发光二极管	LED-RED、LED-YELLOW、LED-GREEN	模拟红、黄、绿交通信号灯
3	电阻	RES	LED 限流保护
4	晶振	CRYSTAL	提供单片机运行时钟
5	电容	CAP / CAP-ELEC	晶振匹配、复位和滤波
6	按键	BUTTON	复位输入和 INT0 应急输入
7	电源端子	POWER / GROUND	提供 +5V 和公共地

3 软件设计与程序流程

3.1 交通灯状态表

表 3 正常循环与应急状态设计

状态	P1 输出值	灯态说明	延时参数
初始/应急	09H	东西红灯，南北红灯	R5=5 或 R5=10
S1	11H	东西红灯，南北绿灯	R5=15
S1 闪烁间隔	01H	东西红灯，南北绿灯熄灭	R5=2
S2	21H	东西红灯，南北黄灯	R5=5
S3	0AH	南北红灯，东西绿灯	R5=15
S3 闪烁间隔	08H	南北红灯，东西绿灯熄灭	R5=2
S4	0CH	南北红灯，东西黄灯	R5=5

3.2 程序流程图

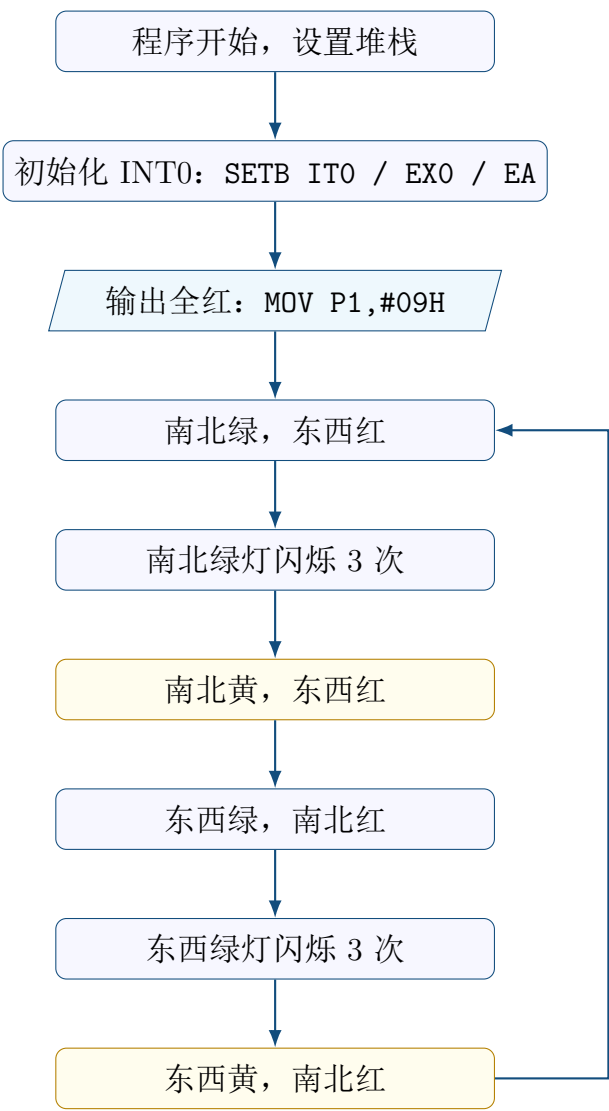


图 6 交通灯主程序流程图

3.3 INT0 应急中断流程

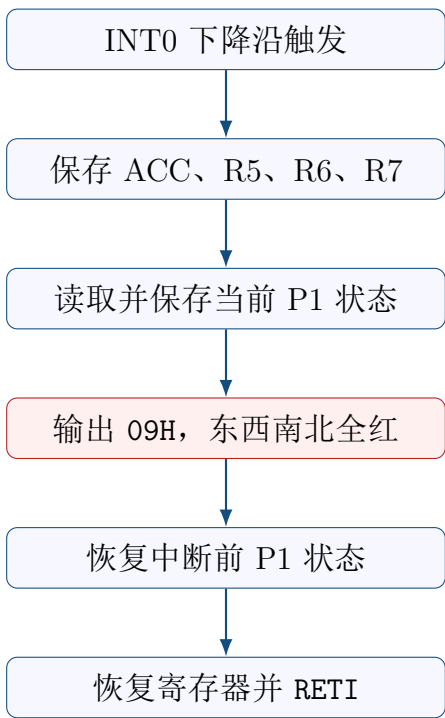


图 7 INT0 应急中断处理流程图

3.4 交通灯状态时序

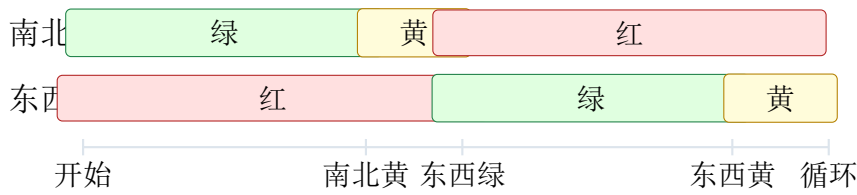


图 8 正常交通灯状态时序示意图

3.5 完整实验代码

Listing 1 03_traffic_light.A51 完整源码

```
1 ;=====
2 ; AT89C51 traffic light experiment with INT0 emergency mode
3 ; Compiler: ASEM-51 (Proteus)
4 ;=====
5
6 $NOMOD51
7 $INCLUDE (8051.MCU)
```

```

8
9 ;=====
10 ; Interrupt vectors
11 ;=====
12
13     ORG 0000H
14     LJMP START
15
16     ORG 0003H           ; INTO external interrupt vector
17     LJMP INTO_ISR
18
19 ;=====
20 ; Main program
21 ;=====
22
23     ORG 0100H
24
25 START:
26     MOV SP,#60H
27
28     SETB P3.2           ; keep INTO input latch high
29     SETB ITO            ; falling-edge trigger for INTO
30     SETB EX0            ; enable INTO
31     SETB EA             ; enable global interrupt
32
33     ; Initial state: all red
34     MOV P1,#09H         ; east-west red + north-south red
35     MOV R5,#5
36     ACALL DELAY
37
38 MAIN:
39     ; State 1: north-south green, east-west red
40     MOV P1,#11H         ; P1.4 + P1.0
41     MOV R5,#15
42     ACALL DELAY
43
44     ; North-south green flashes three times
45     MOV P1,#01H
46     MOV R5,#2
47     ACALL DELAY

```

```
48
49     MOV P1,#11H
50     MOV R5,#2
51     ACALL DELAY
52
53     MOV P1,#01H
54     MOV R5,#2
55     ACALL DELAY
56
57     MOV P1,#11H
58     MOV R5,#2
59     ACALL DELAY
60
61     MOV P1,#01H
62     MOV R5,#2
63     ACALL DELAY
64
65     MOV P1,#11H
66     MOV R5,#2
67     ACALL DELAY
68
69     ; North-south yellow, east-west red
70     MOV P1,#21H          ; P1.5 + P1.0
71     MOV R5,#5
72     ACALL DELAY
73
74     ; State 3: east-west green, north-south red
75     MOV P1,#0AH          ; P1.1 + P1.3
76     MOV R5,#15
77     ACALL DELAY
78
79     ; East-west green flashes three times
80     MOV P1,#08H
81     MOV R5,#2
82     ACALL DELAY
83
84     MOV P1,#0AH
85     MOV R5,#2
86     ACALL DELAY
87
```

```

88      MOV P1,#08H
89      MOV R5,#2
90      ACALL DELAY
91
92      MOV P1,#0AH
93      MOV R5,#2
94      ACALL DELAY
95
96      MOV P1,#08H
97      MOV R5,#2
98      ACALL DELAY
99
100     MOV P1,#0AH
101     MOV R5,#2
102     ACALL DELAY
103
104     ; East-west yellow, north-south red
105     MOV P1,#0CH      ; P1.2 + P1.3
106     MOV R5,#5
107     ACALL DELAY
108
109     SJMP MAIN
110
111     ;=====
112     ; INTO emergency interrupt
113     ; After the emergency key is pressed, all directions stay red briefly.
114     ;=====
115
116     INTO_ISR:
117         PUSH ACC
118         PUSH 05H
119         PUSH 06H
120         PUSH 07H
121
122         MOV A,P1
123         MOV 30H,A      ; save current P1 state
124
125         MOV P1,#09H    ; emergency state: all red
126         MOV R5,#10
127         ACALL DELAY

```

```
128
129      MOV A,30H
130      MOV P1,A           ; restore previous traffic state
131
132      POP 07H
133      POP 06H
134      POP 05H
135      POP ACC
136      RETI
137
138      ;=====
139      ; Delay subroutine
140      ; R5 = delay multiplier
141      ;=====
142
143      DELAY:
144      D1:
145          MOV R6,#200
146      D2:
147          MOV R7,#250
148      D3:
149          DJNZ R7,D3
150          DJNZ R6,D2
151          DJNZ R5,D1
152          RET
153
154      ;=====
155      END
```

3.6 关键代码说明

表 4 关键指令与功能说明

指令或语句	功能说明
ORG 0003H / LJMP INT0_ISR	设置 INT0 外部中断入口，使应急按钮触发后进入中断服务程序。
SETB ITO	配置 INT0 为下降沿触发，适合按键输入产生的触发信号。
SETB EX0 / SETB EA	分别打开 INT0 中断允许位和总中断允许位。
MOV P1,#11H	输出东西红灯、南北绿灯，对应南北方向通行。
MOV P1,#21H	输出东西红灯、南北黄灯，对应南北方向由通行转为停止的过渡状态。
MOV P1,#0AH	输出南北红灯、东西绿灯，对应东西方向通行。
MOV P1,#09H	输出东西红灯、南北红灯，用于初始状态和应急全红状态。
PUSH / POP / RETI	完成中断现场保护、现场恢复和中断返回。

4 调试过程与实验结果

4.1 调试过程记录

1. 在 Proteus 中完成 AT89C51、供电、复位、晶振、交通灯 LED 和 INT0 应急按键连接。
2. 将交通灯汇编程序加载到 AT89C51，检查复位入口和 INT0 中断入口地址是否正确。
3. 启动仿真，观察初始状态是否为东西、南北全红。
4. 连续观察正常循环，确认南北通行、南北黄灯、东西通行、东西黄灯顺序正确。
5. 在不同灯态下按下 INT0 应急按键，检查是否立即进入全红状态，并在延时后恢复原灯态。

4.2 结果验证表

表 5 交通灯仿真结果记录

序号	测试内容	预期现象	结果
1	上电复位	东西红灯、南北红灯点亮	符合
2	正常循环第一阶段	南北绿灯亮，东西红灯亮	符合
3	南北通行结束提示	南北绿灯闪烁后切换为南北黄灯	符合
4	正常循环第二阶段	东西绿灯亮，南北红灯亮	符合
5	东西通行结束提示	东西绿灯闪烁后切换为东西黄灯	符合
6	INT0 应急按键	立即进入全红，延时后恢复中断前灯态	符合

4.3 程序正确性分析

程序采用固定状态表输出 P1 端口值，逻辑关系清晰：任意时刻两个方向不会同时出现绿灯，从而避免交通冲突。INT0 中断服务程序在进入时保存 ACC、R5、R6、R7，并额外保存 P1 当前值，确保应急全红结束后可以恢复原来的交通灯显示状态。

5 问题分析与改进建议

- 1. 当前延时由软件循环完成，时间精度受晶振频率和指令周期影响；实际系统中可改用定时器中断获得更稳定的秒级计时。
- 2. INT0 使用机械按键时可能产生抖动，硬件上可加入 RC 滤波，软件上可在中断入口增加短延时消抖。
- 3. 交通灯状态目前采用顺序代码展开，若后续扩展倒计时数码管，可将状态、输出值和延时参数整理为表驱动结构。

6 实验结论

本实验完成了基于 AT89C51 的交通灯控制系统设计与仿真。程序通过 P1 口输出东西、南北两个方向的红黄绿灯控制信号，实现了正常通行、绿灯闪烁、黄灯过渡和循环切换；同时利用 INT0 外部中断实现应急全红功能。仿真结果表明，硬件连接、端口输出和中断处理均满足实验要求。



杭州电子科技大学

实验四 LED 数码管显示实验报告

课程 微机原理与接口技术

学院 自动化学院

专业 自动化（新工科试验班）

学号 24061428

学生姓名 张跃哲

指导教师 金朝阳

完成日期 2026 年 6 月 22 日

目录

1	实验要求与实验目的	1
1.1	实验要求	1
1.2	实验目的	1
1.3	总体设计思路	1
2	硬件电路设计	1
2.1	单片机最小系统	2
2.2	SN74HC573 扩展 P0 口	2
2.3	共阴极 LED 动态显示	3
2.4	Proteus 仿真电路	3
3	软件设计	4
3.1	程序设计思路	4
3.2	显示扫描流程	4
3.3	显示字符与段码表	5
3.4	完整源程序	6
4	仿真调试与结果分析	9
4.1	调试步骤	9
4.2	仿真现象	9
4.3	问题分析	9
5	实验总结	10

1 实验要求与实验目的

1.1 实验要求

本实验要求利用 AT89C51 单片机和两片 SN74HC573 数据锁存器完成 LED 数码管动态显示。由于 AT89C51 的 P0 口直接驱动能力和端口数量有限，本实验把 P0.0–P0.7 作为复用数据总线，分别送入两片 SN74HC573：一片锁存段码，另一片锁存位码。P2.6 用于控制段码锁存器的 LE 端，P2.7 用于控制位选锁存器的 LE 端。系统复位后，在 8 位共阴极 LED 数码管上动态显示 “HELLO-88”。

完整电路需要包含单片机供电、复位电路、晶振电路、P0 口上拉电阻、两片 SN74HC573 锁存电路和 8 位共阴极 LED 数码管显示电路，并完成程序编写与 Proteus 仿真调试。

1.2 实验目的

- 1. 了解八段共阴极 LED 数码管的段选、位选和动态扫描显示原理。
- 2. 掌握利用 AT89C51 并行 I/O 口驱动 LED 数码管的方法。
- 3. 掌握 SN74HC573 锁存器扩展 P0 口、分时输出段码和位码的接口设计方法。
- 4. 熟悉 ASEM-51 汇编程序结构、段码表查表和显示缓冲区设计。

1.3 总体设计思路

系统以 AT89C51 为控制核心，P0 口作为 8 位复用数据总线。输出段码时，程序把段码送到 P0，再通过 P2.6 给段码锁存器一个锁存脉冲；输出位码时，程序把位选码送到 P0，再通过 P2.7 给位选锁存器一个锁存脉冲。由于两片锁存器的输出会保持最近一次锁存的数据，单片机只需按位快速刷新，便可以利用视觉暂留让 8 位数码管看起来同时点亮。

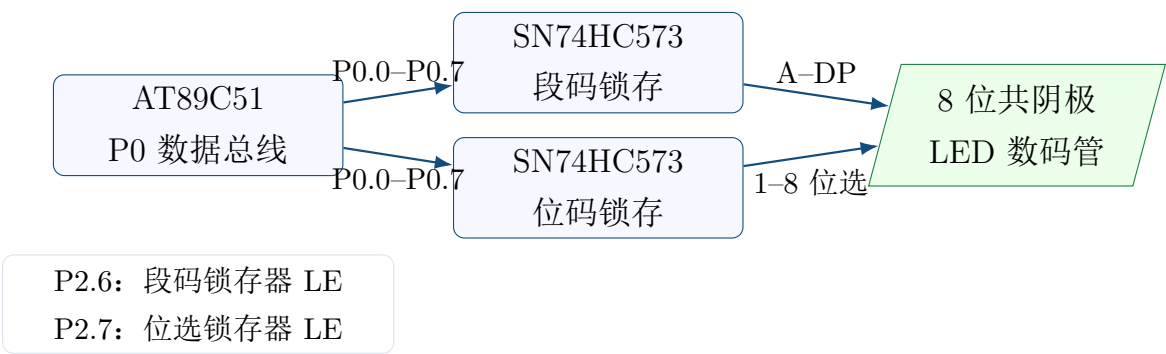
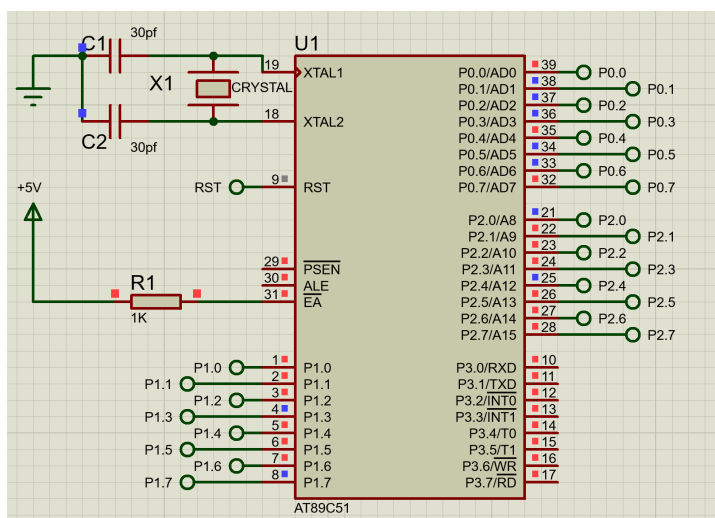


图 1 实验四 LED 数码管显示系统结构

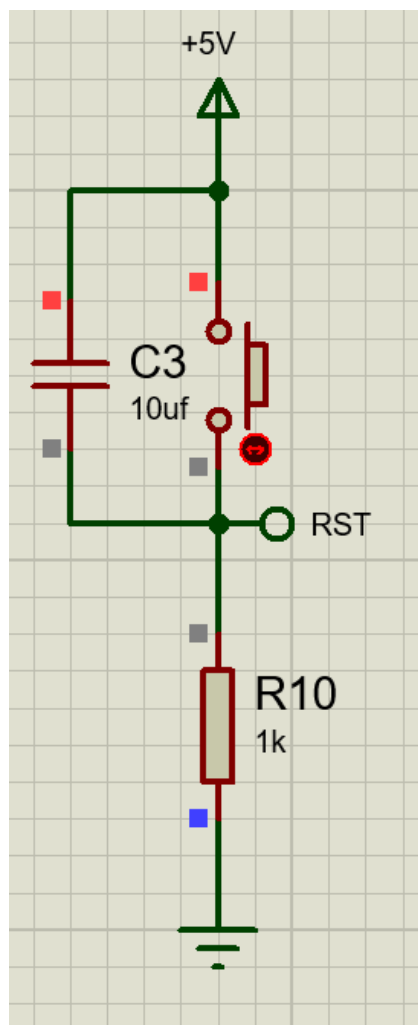
2 硬件电路设计

2.1 单片机最小系统

AT89C51 采用 +5V 供电。XTAL1 与 XTAL2 外接晶振和两个 30pF 电容，构成单片机时钟电路。EA 引脚接高电平，使单片机从片内程序存储器取指。RST 引脚连接复位网络，上电时由电容充电形成复位脉冲，按下复位按钮时也可以使程序重新从 0000H 开始执行。



(a) AT89C51 供电与晶振电路



(b) 复位电路

图 2 单片机基础外围电路

2.2 SN74HC573 扩展 P0 口

AT89C51 的 P0 口为漏极开路结构，作为普通 I/O 使用时需要外接上拉电阻。本实验中 P0.0-P0.7 同时连接两片 SN74HC573 的 D0-D7 输入端，P0 上的数据由 P2.6、P2.7 分别锁存到不同的输出通道。两片 SN74HC573 的 OE 接地，使输出始终允许。

段码锁存器的 Q0-Q7 连接数码管 A、B、C、D、E、F、G、DP 段；位码锁存器的 Q0-Q7 连接数码管第 1-8 位公共端。程序先关闭所有位选，再送段码和位码，可以减小动态扫描时的重影。

2.3 共阴极 LED 动态显示

共阴极数码管的段信号为高电平有效，位选公共端为低电平有效。程序中的位码从 0FEH 开始，表示只有最低位为 0，即选通第 1 位；随后通过 RL A 左移，依次得到 0FDH、0FBH、0F7H 等位选码，从左到右扫描 8 位数码管。

表 1 主要端口分配

单片机端口	连接对象	功能说明
P0.0–P0.7	两片 SN74HC573 的 D0–D7	复用输出段码和位码
P2.6	段码锁存器 LE	锁存 A–DP 段码
P2.7	位码锁存器 LE	锁存第 1–8 位位选码
EA	+5V	片内程序存储器有效
RST	复位网络	上电和按键复位
XTAL1/XTAL2	晶振与 30pF 电容	提供单片机工作时钟

2.4 Proteus 仿真电路

图 3 为实验四 Proteus 仿真关键电路。图中 P0 口通过电阻排上拉后连接两片 74HC573，两片锁存器分别连接数码管的段线和位选线。运行程序后，数码管能够动态显示“HELLO-88”，其中字母 O 由七段数码管中的数字 0 近似表示。

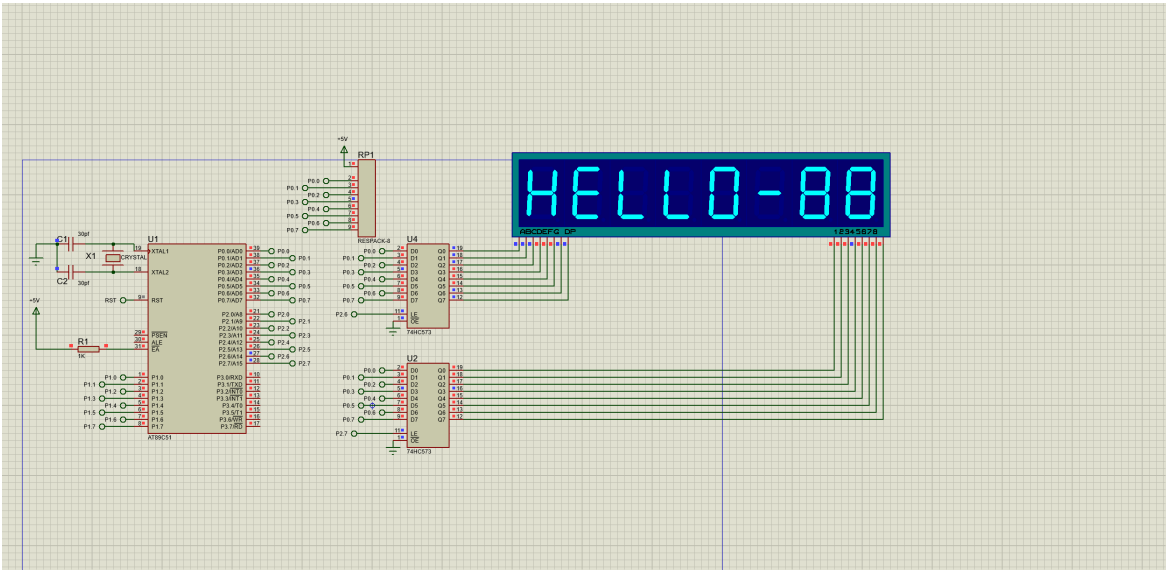


图 3 实验四 LED 数码管显示关键仿真图

3 软件设计

3.1 程序设计思路

程序首先初始化堆栈指针和显示缓冲区。显示缓冲区从内部 RAM 的 50H 开始，共 8 个单元，依次保存 H、E、L、L、O、-、8、8 的段码索引。主循环不断调用显示扫描逻辑，每次从 50H 开始读取显示缓冲单元，查段码表得到真实段码，然后送入段码锁存器；再把当前位选码送入位码锁存器。每显示一位后进行短延时，再扫描下一位。

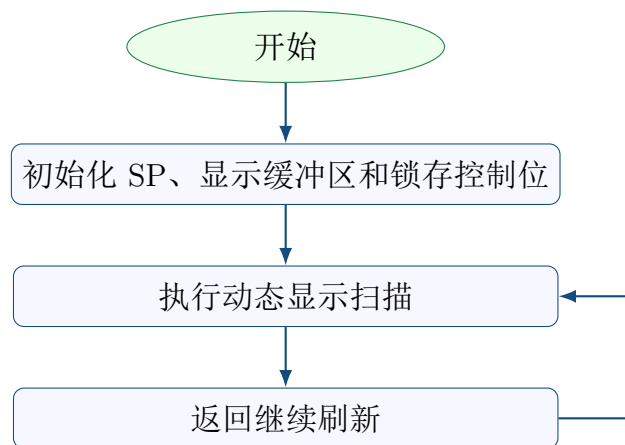


图 4 主程序流程图

3.2 显示扫描流程

显示扫描部分的核心是“先消隐、再送段码、再送位码”。这样可以避免上一位的位选仍然有效时段码发生变化，从而减小重影。扫描码初值为 0FEH，每轮扫描后左移一位。当最高位已经移出扫描范围时，程序关闭段码和位码后重新开始新一轮扫描。

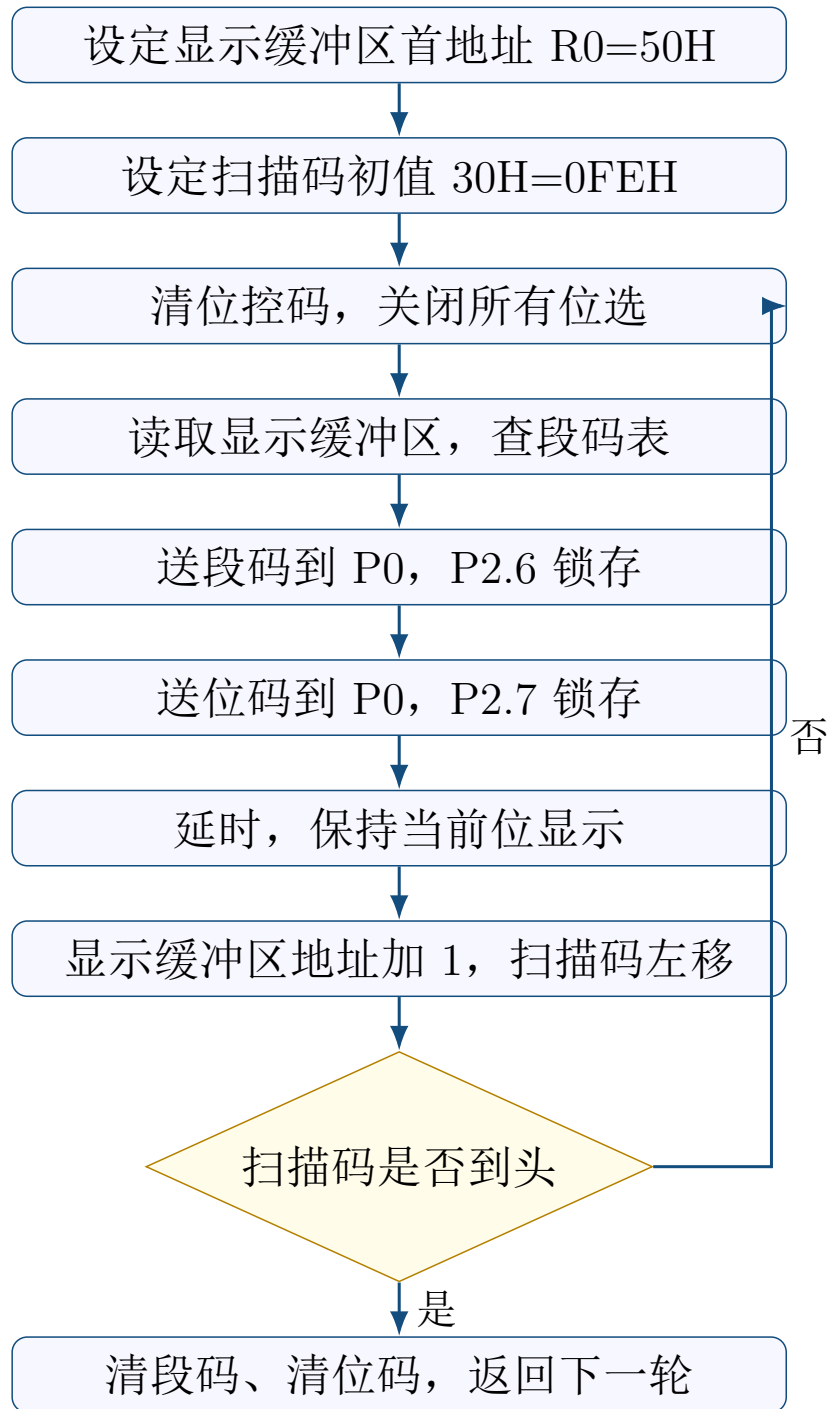


图 5 显示子程序流程图

3.3 显示字符与段码表

七段数码管不能像点阵屏一样显示标准字母，因此本实验使用可识别的近似字符。程序中显示缓冲区保存的是段码表索引，实际显示时再通过 `MOVC A, @A+DPTR` 查表得到段码。

表 2 HELLO-88 显示缓冲区与段码

显示位置	缓冲区地址	缓冲区索引	查表后段码
1	50H	11H	76H，显示 H
2	51H	0EH	79H，显示 E
3	52H	12H	38H，显示 L
4	53H	12H	38H，显示 L
5	54H	00H	3FH，显示 0 近似 O
6	55H	13H	40H，显示 -
7	56H	08H	7FH，显示 8
8	57H	08H	7FH，显示 8

3.4 完整源程序

Listing 1 04_lcd.A51 完整源程序

```
1 ;=====
2 ; Experiment 4: LED数码管显示 (74HC573动态扫描)
3 ; Compiler: ASEM-51 (Proteus)
4 ;
5 ; P0.0-P0.7 -> 两片74HC573的D0-D7
6 ; P2.6      -> 段码锁存器 LE
7 ; P2.7      -> 位选锁存器 LE
8 ;=====
9
10 $NOMOD51
11 $INCLUDE (8051.MCU)
12
13     ORG 0000H
14     LJMP MAIN
15
16     ORG 0100H
17 MAIN:
18     MOV SP, #5FH
19
20     ; 显示缓冲区: H E L L O - 8 8
```

```
21      MOV 50H, #11H      ; H
22      MOV 51H, #0EH      ; E
23      MOV 52H, #12H      ; L
24      MOV 53H, #12H      ; L
25      MOV 54H, #00H      ; 0, 用数字0近似
26      MOV 55H, #13H      ; -
27      MOV 56H, #08H      ; 8
28      MOV 57H, #08H      ; 8
29
30      CLR P2.6
31      CLR P2.7
32
33 DISP:
34      MOV R0, #50H
35      MOV 30H, #0FEH
36
37 NEXT:
38      ; 先关闭所有位, 防止重影
39      MOV P0, #0FFH
40      SETB P2.7
41      NOP
42      CLR P2.7
43
44      ; 查段码并锁存
45      MOV DPTR, #TABLE
46      MOV A, @R0
47      MOVC A, @A+DPTR
48      MOV P0, A
49      SETB P2.6
50      NOP
51      CLR P2.6
52
53      ; 送位码并锁存
54      MOV A, 30H
55      MOV P0, A
56      SETB P2.7
57      NOP
58      CLR P2.7
59
60      ; 延时
```

```

61      MOV R6, #02H
62 DLY1:  MOV R7, #00H
63 DLY2:  DJNZ R7, DLY2
64      DJNZ R6, DLY1
65
66      ; 下一位
67      INC R0
68      MOV A, 30H
69      JNB ACC.7, DONE
70      RL A
71      MOV 30H, A
72      AJMP NEXT
73
74 DONE:
75      ; 先关闭位选, 再清段码
76      MOV P0, #0FFH
77      SETB P2.7
78      NOP
79      CLR P2.7
80
81      MOV P0, #00H
82      SETB P2.6
83      NOP
84      CLR P2.6
85
86      LJMP DISP
87
88 TABLE:
89      ; 0 1 2 3 4
90      DB 3FH, 06H, 5BH, 4FH, 66H
91      ; 5 6 7 8 9
92      DB 6DH, 7DH, 07H, 7FH, 6FH
93      ; A b C d E
94      DB 77H, 7CH, 39H, 5EH, 79H
95      ; F blank H L -
96      DB 71H, 00H, 76H, 38H, 40H
97
98      END

```

4 仿真调试与结果分析

4.1 调试步骤

1. 检查 AT89C51 的 +5V 电源、EA、晶振和复位电路，确认程序能够从复位地址启动。
2. 检查 P0 口是否接上拉电阻排，避免 P0 开漏输出导致高电平异常。
3. 检查两片 SN74HC573 的 D0–D7 是否都接到 P0.0–P0.7，OE 是否接地。
4. 检查 P2.6 是否接段码锁存器 LE，P2.7 是否接位码锁存器 LE。
5. 单独测试第一位显示 0，确认段线顺序和第一位位选正确。
6. 加载完整程序，观察 8 位数码管是否稳定显示 “HELLO-88”。

4.2 仿真现象

在 Proteus 中运行程序后，8 位共阴极数码管动态显示 “HELLO-88”。由于七段数码管的显示能力有限，字母 O 使用数字 0 的段码近似显示，因此实际视觉效果为 “HELL0-88”。H、E、L、-、8 等字符均通过段码表实现，显示稳定，说明 P0 扩展、锁存控制和动态扫描逻辑均能正常工作。

4.3 问题分析

本实验调试中最容易出现的问题是段码和位码接线顺序不一致。若段码锁存器 Q0–Q7 没有按 A、B、C、D、E、F、G、DP 顺序连接，字符会显示成错误形状；若位码锁存器 Q0–Q7 与第 1–8 位顺序不一致，则字符顺序会颠倒或错位。此外，P0 口必须接上拉电阻，否则高电平输出不可靠。通过固定段码 3FH、固定位码 0FEH 单独点亮第一位显示 0，可以快速判断基础段选和位选连接是否正确。

表 3 常见故障与排查方法

现象	可能原因	排查方法
数码管完全不亮	位选未打开、OE 未接地或程序未运行	检查 P2.7、74HC573 OE、晶振复位
字符形状错误	段线 A-DP 顺序接错	固定显示 0、H、E 分别测试段码
字符顺序错误	位选 1-8 顺序接错	固定段码后逐位测试位码
显示重影	未先关闭位选或扫描延时过长	先消隐再送段码，适当调整延时
显示亮度偏低	扫描频率过高或驱动电流不足	调整延时，检查限流和驱动连接

5 实验总结

本实验完成了基于 AT89C51、两片 SN74HC573 和 8 位共阴极 LED 数码管的动态显示系统设计。硬件上，利用 P0 口作为复用数据总线，通过 P2.6 和 P2.7 分别控制段码锁存器和位码锁存器，实现了 P0 口扩展；软件上，采用显示缓冲区、段码查表和位码左移扫描的方式，使系统复位后能够稳定显示“HELLO-88”。通过本实验，进一步理解了 LED 数码管动态显示的时间复用思想，也掌握了锁存器扩展单片机 I/O 口的基本方法。



杭州电子科技大学

实验五 波形发生器实验

课 程 微机原理与接口技术

学 院 自动化学院

专 业 自动化 (新工科试验班)

学 号 24061428

学生姓名 张跃哲

指导教师 金朝阳

完成日期 2026 年 6 月 1 日

目录

1	实验要求与实验目的	1
1.1	实验要求	1
1.2	实验目的	1
1.3	总体设计思路	1
2	Proteus 原理图与硬件电路说明	2
2.1	完整原理图	2
2.2	单片机最小系统	3
2.3	DAC0832 与模拟输出电路	5
2.4	波形选择与 LED 指示电路	6
2.5	频率选择电路	7
2.6	主要元件清单	8
3	软件设计与程序流程	8
3.1	端口分配	8
3.2	主程序流程	9
3.3	四种波形生成方法	9
3.4	正弦波查表流程	10
3.5	延时与频率控制	10
3.6	完整实验代码	10
3.7	关键代码说明	17
4	调试过程与实验结果	17
4.1	调试步骤	17
4.2	四种输出波形	18
4.3	频率调节对比	18
4.4	结果验证表	19
5	问题分析与改进建议	19
6	实验结论	19

1 实验要求与实验目的

1.1 实验要求

本实验要求在 Proteus 中搭建以 AT89C51 和 DAC0832 为核心的波形发生器。单片机通过并行口向 DAC0832 输出 8 位数字量，经运算放大器转换为模拟电压，并在示波器上观察输出波形。基本要求如下：

1. 使用 AT89C51 的 P0 口连接 DAC0832 的 DI0–DI7，完成 8 位数字量到模拟量的转换。
2. 使用 P1.0–P1.3 的拨码开关选择方波、锯齿波、三角波和正弦波四种输出波形。
3. 使用 P1.4–P1.7 的 LED 指示当前选中的波形，LED 采用低电平有效方式驱动。
4. 使用 P2 口作为频率选择输入，通过改变 P2 口拨码值调节软件延时，从而改变输出波形频率。

在完成基本功能的基础上，实验还包含以下拓展要求：

1. 在 Proteus 中完整呈现单片机最小系统、DAC0832 接口、运放输出、波形选择和频率选择电路。
2. 对四种波形分别进行仿真截图，证明输出波形形状与程序设计相符。
3. 对不同 P2 输入下的波形频率进行对比，说明频率控制功能有效。

1.2 实验目的

1. 掌握 AT89C51 并行 I/O 口输出数字量、读取拨码输入以及驱动 LED 指示电路的方法。
2. 理解 DAC0832 的数字量输入、参考电压和运算放大器电流-电压转换输出原理。
3. 掌握用软件查表和循环计数生成周期波形的的方法，并分析软件延时对频率的影响。
4. 提高 Proteus 环境下单片机、D/A 转换器和虚拟示波器联合调试的能力。

1.3 总体设计思路

系统以 AT89C51 为控制核心，P0 口作为波形数据总线，直接把 8 位采样值送入 DAC0832。DAC0832 工作在直通转换方式，片选和写控制信号固定为有效状态，P0 口数据变化后即可在模拟输出端体现为对应电压。P1 口低四位连接波形选择拨码开关，程序每输出一个周期后重新扫描按键；P1 口高四位连接 LED，用于指示当前波形。P2 口连接 8 位拨码开关作为频率控制量，程序将其取反后作为延时循环次数，因此 P2 值越大，延时越短，输出频率越高。

2 Proteus 原理图与硬件电路说明

2.1 完整原理图

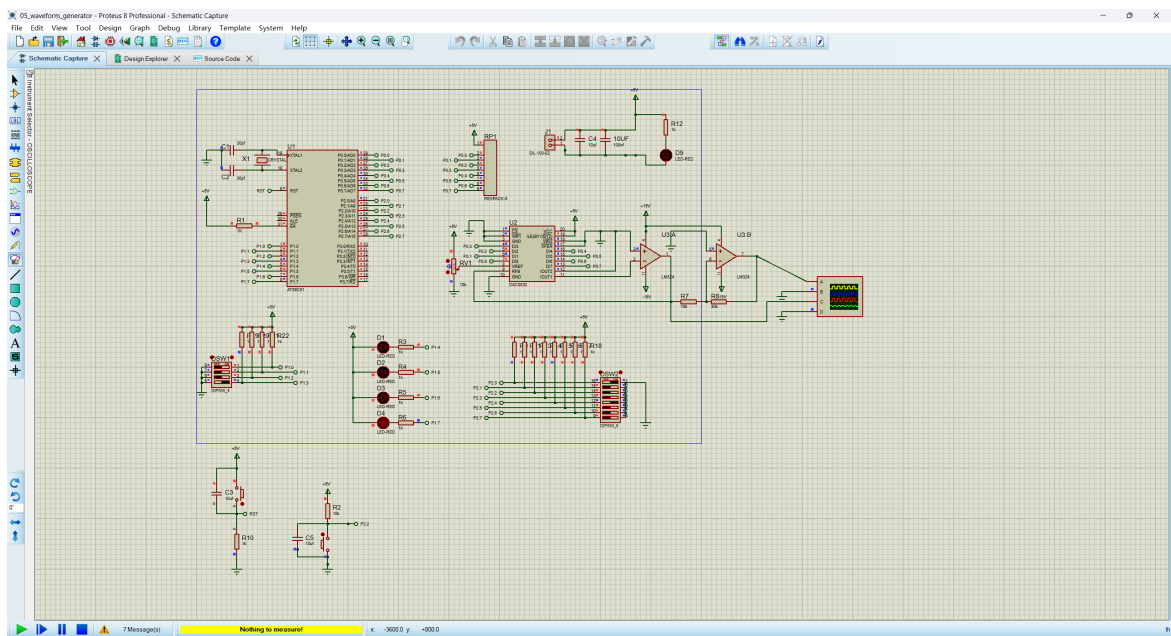


图 1 实验五波形发生器 Proteus 完整原理图

完整电路由 AT89C51 最小系统、DAC0832 数模转换模块、LM324 运算放大器输出模块、P1 口波形选择与 LED 指示模块、P2 口频率选择模块组成。数字部分使用 +5V 供电，运放部分采用正负电源供电，以便输出端能够获得较好的模拟波形摆幅。

2.2 单片机最小系统

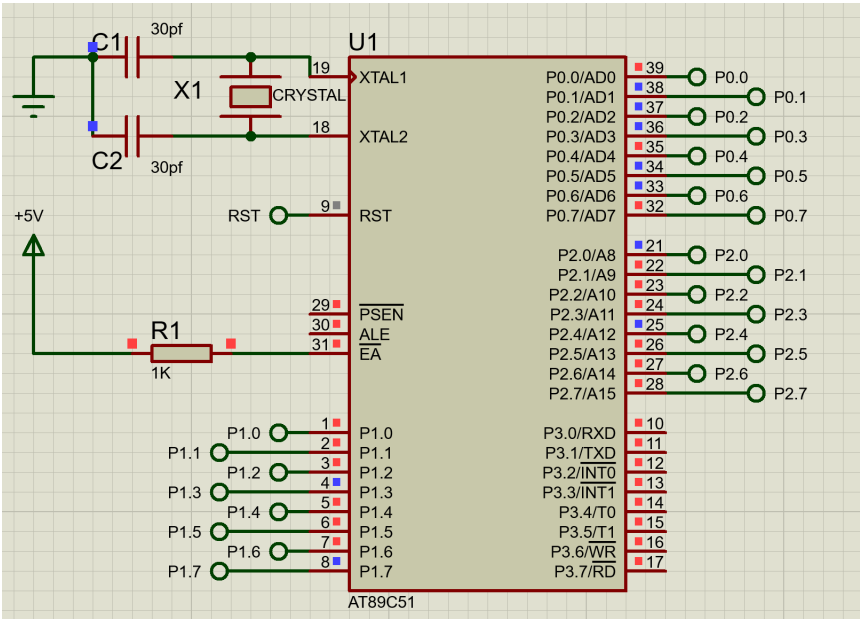


图 2 AT89C51 供电与 12MHz 晶振电路

AT89C51 的 VCC 与 GND 接入 +5V 电源和公共地，XTAL1、XTAL2 外接 12MHz 晶振，并配合两个 30pF 电容构成时钟振荡电路。12MHz 晶振下，8051 一个机器周期约为 1 μ s，便于通过软件循环估算延时时间。

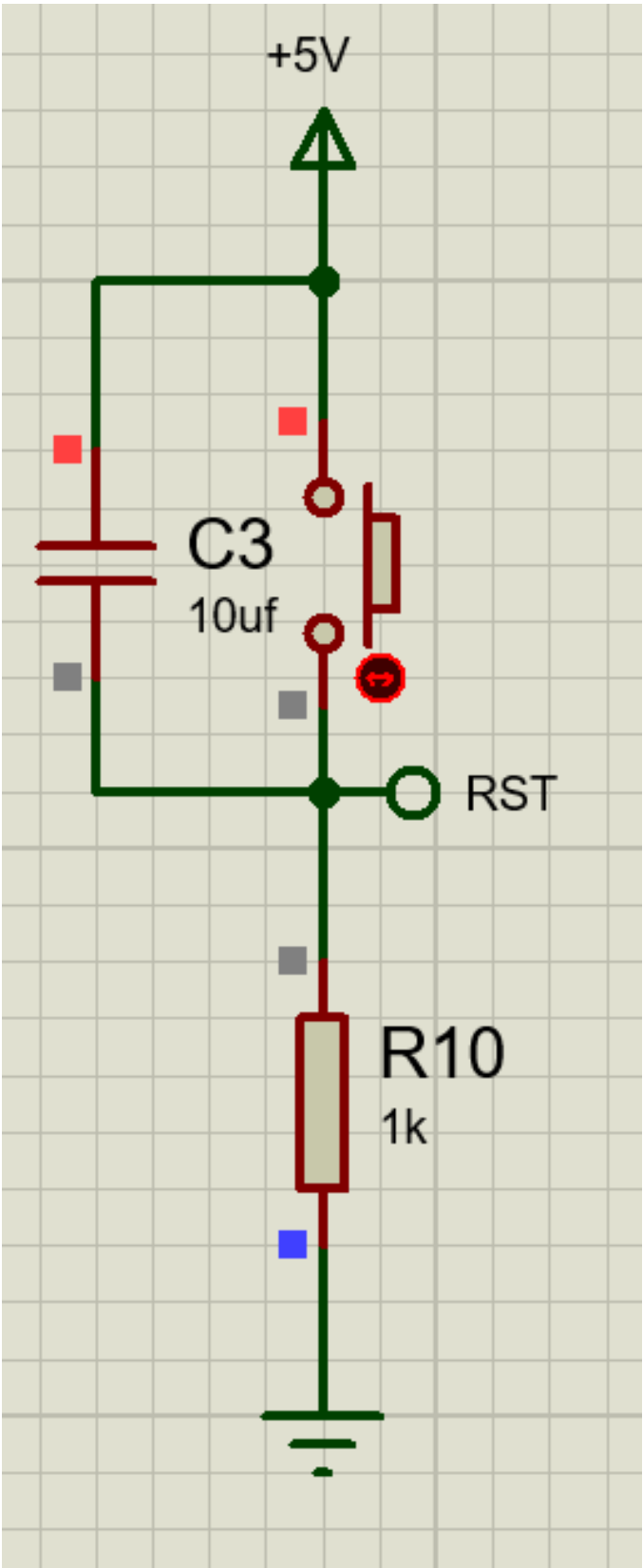


图 3 单片机复位电路

复位电路由按键、电阻和电容构成。上电时电容充电产生短暂高电平，按下复位键时 RST 引脚被拉到有效电平，使程序从 0000H 复位向量重新开始执行。

2.3 DAC0832 与模拟输出电路

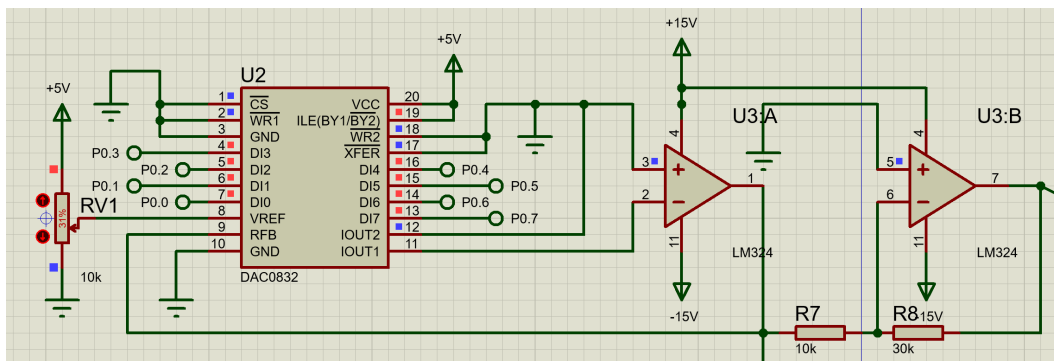


图 4 DAC0832 与 LM324 运放输出电路

DAC0832 的 DI0–DI7 与 AT89C51 的 P0.0–P0.7 相连，作为 8 位并行输入数据总线。电路中 CS、WR1、WR2、XFER 等控制端固定为有效状态，使 DAC0832 处于直通工作方式。单片机写入 P0 的数据不需要额外片选脉冲，即可进入 D/A 转换链路。参考电压由电位器调节，输出电流经 LM324 运算放大器转换为电压信号，最终送到示波器或模拟波形图进行观察。

2.4 波形选择与 LED 指示电路

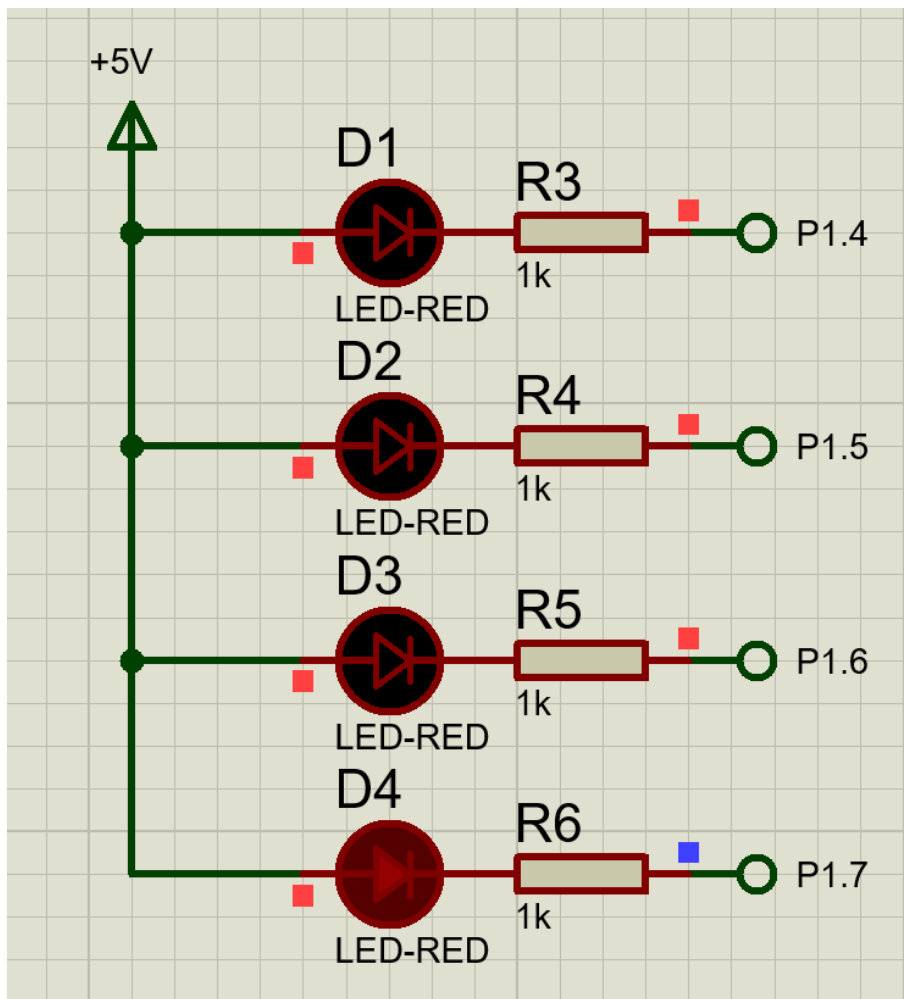


图 5 P1 口波形选择拨码开关与 LED 指示电路

P1.0–P1.3 分别对应方波、锯齿波、三角波和正弦波选择输入，拨码开关闭合时相应引脚为低电平。P1.4–P1.7 分别连接四个 LED，程序输出低电平点亮相应指示灯。由于 P1 口同一端口同时承担输入和输出功能，程序在点亮 LED 时保持低四位为高电平，避免影响下一次按键扫描。

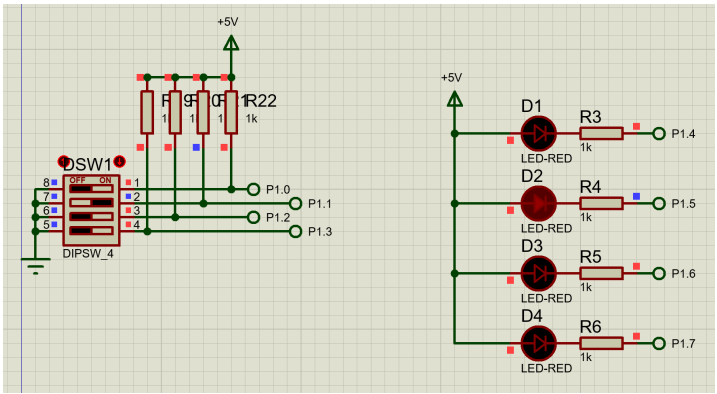


图 6 波形指示 LED 点亮状态

2.5 频率选择电路

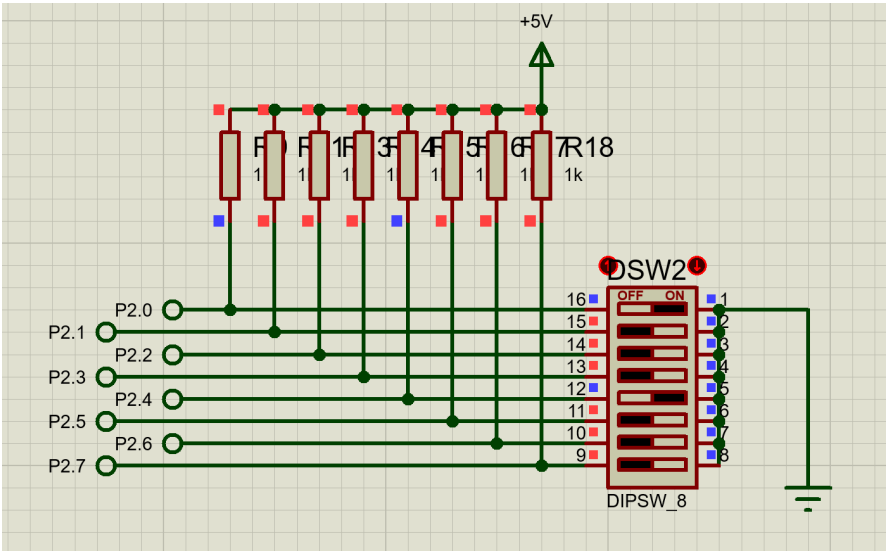


图 7 P2 口 8 位频率选择拨码开关

P2.0–P2.7 连接 DIPSW_8 作为频率输入。程序在延时函数中读取 P2，然后执行取反操作，得到软件延时循环次数。当 P2 值较小时，取反后的延时值较大，波形周期较长；当 P2 值较大时，延时值较小，波形周期缩短。

2.6 主要元件清单

表 1 实验五主要元件与功能

序号	元件名称	型号或规格	功能说明
1	单片机	AT89C51	读取拨码开关，输出波形采样值，控制 LED 指示
2	数模转换器	DAC0832	将 P0 口 8 位数字量转换为模拟电流输出
3	运算放大器	LM324	完成电流-电压转换及模拟电压输出
4	晶振	12MHz	提供单片机运行时钟
5	电位器	POT-HG	调节 DAC0832 参考电压，影响输出幅度
6	拨码开关	DIPSW	P1 选择波形，P2 调节频率
7	LED 与电阻	LED / RES	指示当前输出波形并限制 LED 电流
8	示波器或波形图	OSCILLOSCOPE / GRAPH	观察运放输出端模拟波形

3 软件设计与程序流程

3.1 端口分配

表 2 AT89C51 端口分配表

端口	连接对象	软件作用
P0.0–P0.7	DAC0832 DI0–DI7	输出 8 位波形采样值
P1.0	方波选择开关	低电平有效，返回波形编号 00H
P1.1	锯齿波选择开关	低电平有效，返回波形编号 01H
P1.2	三角波选择开关	低电平有效，返回波形编号 02H
P1.3	正弦波选择开关	低电平有效，返回波形编号 03H
P1.4–P1.7	四个 LED 指示灯	低电平点亮，显示当前波形
P2.0–P2.7	频率选择拨码开关	输入延时调节量，P2 越大频率越高

3.2 主程序流程

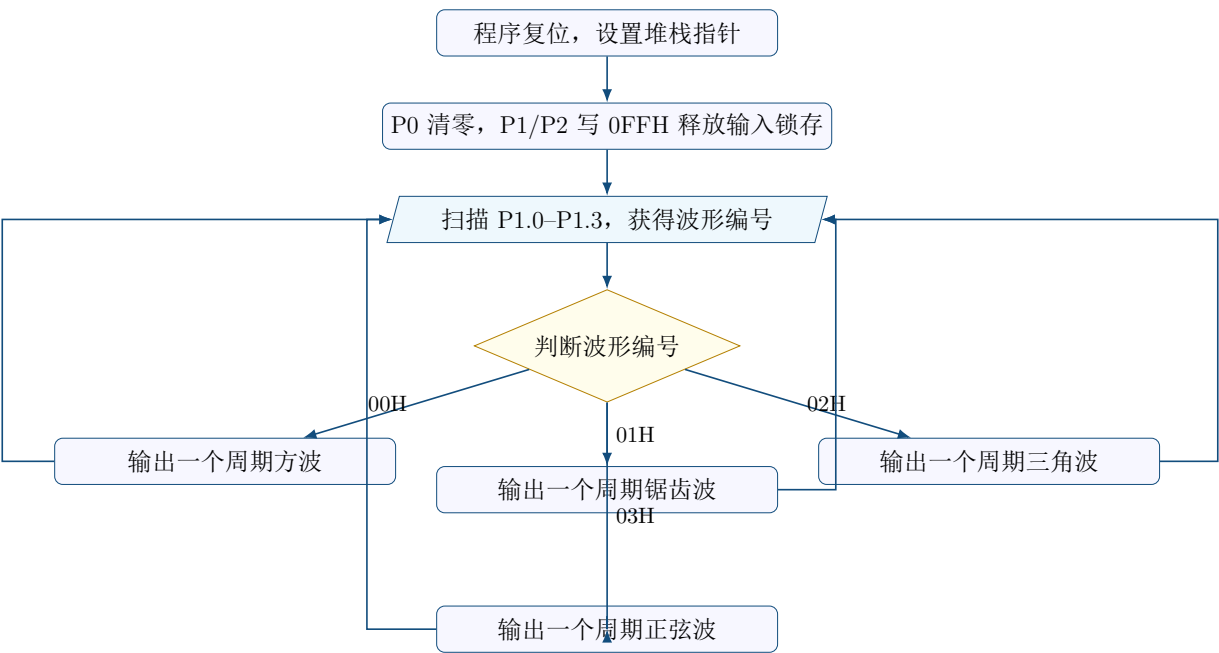


图 8 波形发生器主程序流程图

主程序采用循环扫描方式。每次调用对应波形子程序时只输出一个完整周期，周期结束后返回主循环重新读取 P1 低四位。这样设计的好处是按键切换不需要中断支持，程序结构清晰，且能够在一个周期结束后稳定切换到新的波形。

3.3 四种波形生成方法

表 3 四种波形的软件生成方法

波形	输出规律	程序实现
方波	00H 与 FFH 交替输出	低电平保持半周期，高电平保持半周期，形成高低电平交替
锯齿波	00H 到 FFH 单调递增	累加器从 00H 自增到溢出，每步输出一次并延时
三角波	00H 到 FFH 再回到 00H	先执行递增输出，再执行递减输出，形成线性上升和下降
正弦波	以 80H 为中心上下变化	读取四分之一正弦表，通过对称和取反生成完整周期

3.4 正弦波查表流程

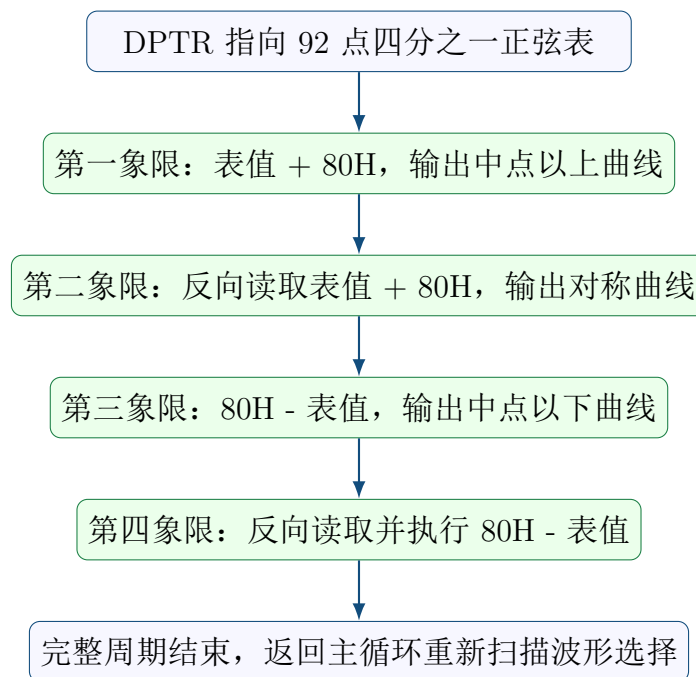


图 9 正弦波查表与对称生成流程图

正弦表中仅保存从 0 到峰值的四分之一周期幅值，范围为 00H–7FH。程序输出时以 80H 作为中心值，前两个象限采用加法生成上半周，后两个象限采用减法生成下半周。与直接存储完整周期相比，这种方法节省了代码空间，同时保证输出曲线平滑。

3.5 延时与频率控制

延时函数 STEP_DELAY 是频率控制的核心。程序读取 P2 后先取反，得到 255 - P2，再作为 R6 的循环次数。若 P2 为 FFH，取反结果为 00H，程序通过 INC A 保证仍有最小延时。其关系可概括为：

$$T_{\text{step}} \propto 255 - P2$$

因此 P2 值越大，每个采样点之间的间隔越短；波形每周期采样点数不变时，周期缩短，频率升高。方波使用 HALF_DELAY 形成半周期延时，锯齿波、三角波和正弦波则在每个采样点后调用 STEP_DELAY。

3.6 完整实验代码

Listing 1 05_waveform_generator.A51 完整源代码

```

1 ;=====
2 ; Experiment 5: AT89C51 + DAC0832 waveform generator
3 ; Compiler: ASEM-51 (Proteus)

```

```

4 ;
5 ; P0.0-P0.7 -> DAC0832 DIO-DI7
6 ; P1.0      -> square wave select, low active
7 ; P1.1      -> sawtooth wave select, low active
8 ; P1.2      -> triangle wave select, low active
9 ; P1.3      -> sine wave select, low active
10 ; P1.4     -> square wave LED, low active
11 ; P1.5     -> sawtooth wave LED, low active
12 ; P1.6     -> triangle wave LED, low active
13 ; P1.7     -> sine wave LED, low active
14 ; P2       -> frequency input, larger value means higher frequency
15 ;=====
16
17 $NOMOD51
18 $INCLUDE (8051.MCU)
19
20 ;=====
21 ; Reset vector
22 ;=====
23
24     ORG 0000H
25     LJMP START
26
27 ;=====
28 ; Main program
29 ;=====
30
31     ORG 0100H
32
33 START:
34     MOV SP,#60H
35     MOV P0,#00H
36     MOV P2,#0FFH      ; release P2 input latch
37     MOV P1,#0FFH      ; release P1.0-P1.3 and turn LEDs off
38
39 MAIN:
40     ACALL SCAN_KEY
41
42     CJNE A,#00H,CHK_SAW
43     ACALL SQUARE_WAVE

```

```

44         SJMP MAIN
45
46 CHK_SAW:
47         CJNE A,#01H,CHK_TRI
48         ACALL SAW_WAVE
49         SJMP MAIN
50
51 CHK_TRI:
52         CJNE A,#02H,CHK_SIN
53         ACALL TRI_WAVE
54         SJMP MAIN
55
56 CHK_SIN:
57         ACALL SINE_WAVE
58         SJMP MAIN
59
60 ;=====
61 ; Scan waveform select key
62 ; Return A:
63 ; 00H square, 01H sawtooth, 02H triangle, 03H sine
64 ; If no key is pressed, square wave is selected by default.
65 ;=====
66
67 SCAN_KEY:
68         MOV A,P1
69         ANL A,#0FH
70         JNB ACC.0,KEY_SQUARE
71         JNB ACC.1,KEY_SAW
72         JNB ACC.2,KEY_TRI
73         JNB ACC.3,KEY_SIN
74
75 KEY_SQUARE:
76         MOV A,#00H
77         RET
78
79 KEY_SAW:
80         MOV A,#01H
81         RET
82
83 KEY_TRI:

```

```

84         MOV A,#02H
85         RET
86
87 KEY_SIN:
88         MOV A,#03H
89         RET
90
91 ;=====
92 ; Waveform output routines
93 ; Each routine outputs one cycle, then returns to MAIN to rescan keys.
94 ;=====
95
96 SQUARE_WAVE:
97         MOV P1,#0EFH      ; P1.4 LED on, P1.0-P1.3 stay high
98         MOV P0,#00H
99         ACALL HALF_DELAY
100        MOV P0,#0FFH
101        ACALL HALF_DELAY
102        RET
103
104 SAW_WAVE:
105        MOV P1,#0DFH      ; P1.5 LED on
106        MOV A,#00H
107 SAW_LOOP:
108        MOV P0,A
109        ACALL STEP_DELAY
110        INC A
111        JNZ SAW_LOOP
112        RET
113
114 TRI_WAVE:
115        MOV P1,#0BFH      ; P1.6 LED on
116        MOV A,#00H
117 TRI_UP:
118        MOV P0,A
119        ACALL STEP_DELAY
120        INC A
121        JNZ TRI_UP
122
123        MOV A,#0FFH

```

```
124 TRI_DOWN:
125     MOV PO,A
126     ACALL STEP_DELAY
127     DEC A
128     JNZ TRI_DOWN
129     MOV PO,#00H
130     ACALL STEP_DELAY
131     RET
132
133 SINE_WAVE:
134     MOV P1,#07FH          ; P1.7 LED on
135
136     MOV DPTR,#SINE_TAB
137     MOV RO,#00H
138 SIN_UP:
139     MOV A,RO
140     MOVC A,@A+DPTR
141     ADD A,#80H
142     MOV PO,A
143     ACALL STEP_DELAY
144     INC RO
145     CJNE RO,#5CH,SIN_UP ; 92 table values
146
147 SIN_BACK_MID:
148     DEC RO
149     MOV A,RO
150     MOVC A,@A+DPTR
151     ADD A,#80H
152     MOV PO,A
153     ACALL STEP_DELAY
154     CJNE RO,#00H,SIN_BACK_MID
155
156     MOV RO,#00H
157 SIN_DOWN:
158     MOV A,RO
159     MOVC A,@A+DPTR
160     MOV B,A
161     MOV A,#80H
162     CLR C
163     SUBB A,B
```

```

164         MOV P0,A
165         ACALL STEP_DELAY
166         INC R0
167         CJNE R0,#5CH,SIN_DOWN
168
169 SIN_BACK_ZERO:
170         DEC R0
171         MOV A,R0
172         MOVC A,@A+DPTR
173         MOV B,A
174         MOV A,#80H
175         CLR C
176         SUBB A,B
177         MOV P0,A
178         ACALL STEP_DELAY
179         CJNE R0,#00H,SIN_BACK_ZERO
180         RET
181
182 ;=====
183 ; Delay routines
184 ; P2 = 255 gives the shortest delay, P2 = 0 gives the longest delay.
185 ;=====
186
187 STEP_DELAY:
188         PUSH ACC
189         MOV A,P2
190         CPL A                ; A = 255 - P2
191         JNZ STEP_SET
192         INC A                ; minimum delay when P2 = 255
193 STEP_SET:
194         MOV R6,A
195 STEP_D1:
196         NOP
197         DJNZ R6,STEP_D1
198         POP ACC
199         RET
200
201 HALF_DELAY:
202         MOV R5,#80
203 HALF_D1:

```

```
204      ACALL STEP_DELAY
205      DJNZ R5, HALF_D1
206      RET
207
208      ;=====
209      ; 92-point quarter sine amplitude table, 0 -> 127.
210      ; One full sine cycle is generated around code 80H.
211      ;=====
212
213 SINE_TAB:
214      DB 00H,02H,04H,07H,09H,0BH,0DH,0FH
215      DB 11H,14H,16H,18H,1AH,1CH,1EH,21H
216      DB 23H,25H,27H,29H,2BH,2DH,2FH,31H
217      DB 33H,35H,37H,39H,3BH,3DH,3FH,41H
218      DB 43H,44H,46H,48H,4AH,4CH,4DH,4FH
219      DB 51H,53H,54H,56H,57H,59H,5BH,5CH
220      DB 5EH,5FH,61H,62H,63H,65H,66H,67H
221      DB 69H,6AH,6BH,6CH,6DH,6EH,6FH,70H
222      DB 71H,72H,73H,74H,75H,76H,77H,78H
223      DB 78H,79H,7AH,7AH,7BH,7BH,7CH,7CH
224      DB 7DH,7DH,7DH,7EH,7EH,7EH,7FH,7FH
225      DB 7FH,7FH,7FH,7FH
226
227      ;=====
228      END
```

3.7 关键代码说明

表 4 关键程序段与功能说明

指令或程序段	功能说明
MOV P0,#00H	初始化 DAC 输入数据，使模拟输出从低电平开始。
MOV P2,#0FFH	向 P2 写 1，释放准双向口输入锁存，便于读取拨码开关。
MOV P1,#0FFH	释放 P1 低四位输入，同时关闭高四位 LED。
ANL A,#0FH	只保留 P1.0–P1.3 的波形选择输入。
JNB ACC.0,KEY_SQUARE	检测低电平有效的方波选择开关。
MOV P1,#0EFH	P1.4 输出低电平，点亮方波指示 LED。
INC A / JNZ SAW_LOOP	锯齿波从 00H 递增到 FFH，溢出后结束一个周期。
MOVC A,@A+DPTR	按偏移量读取程序存储器中的正弦表数据。
CPL A	将 P2 输入取反，把较大的 P2 值转换为较短的软件延时。

4 调试过程与实验结果

4.1 调试步骤

1. 检查 AT89C51 的 VCC、GND、晶振和复位连接，确认单片机能够正常启动。
2. 检查 P0 到 DAC0832 DI0–DI7 的连接顺序，避免位序接反导致波形幅值异常。
3. 确认 DAC0832 控制端处于直通有效状态，LM324 供电和反馈连接正确。
4. 将汇编程序加载到 AT89C51，运行仿真后先观察默认方波输出。
5. 依次切换 P1.0–P1.3 拨码开关，观察四个 LED 指示状态和示波器波形。
6. 保持同一种波形，改变 P2 口拨码值，比较低频、中频和高频时波形周期变化。

4.2 四种输出波形

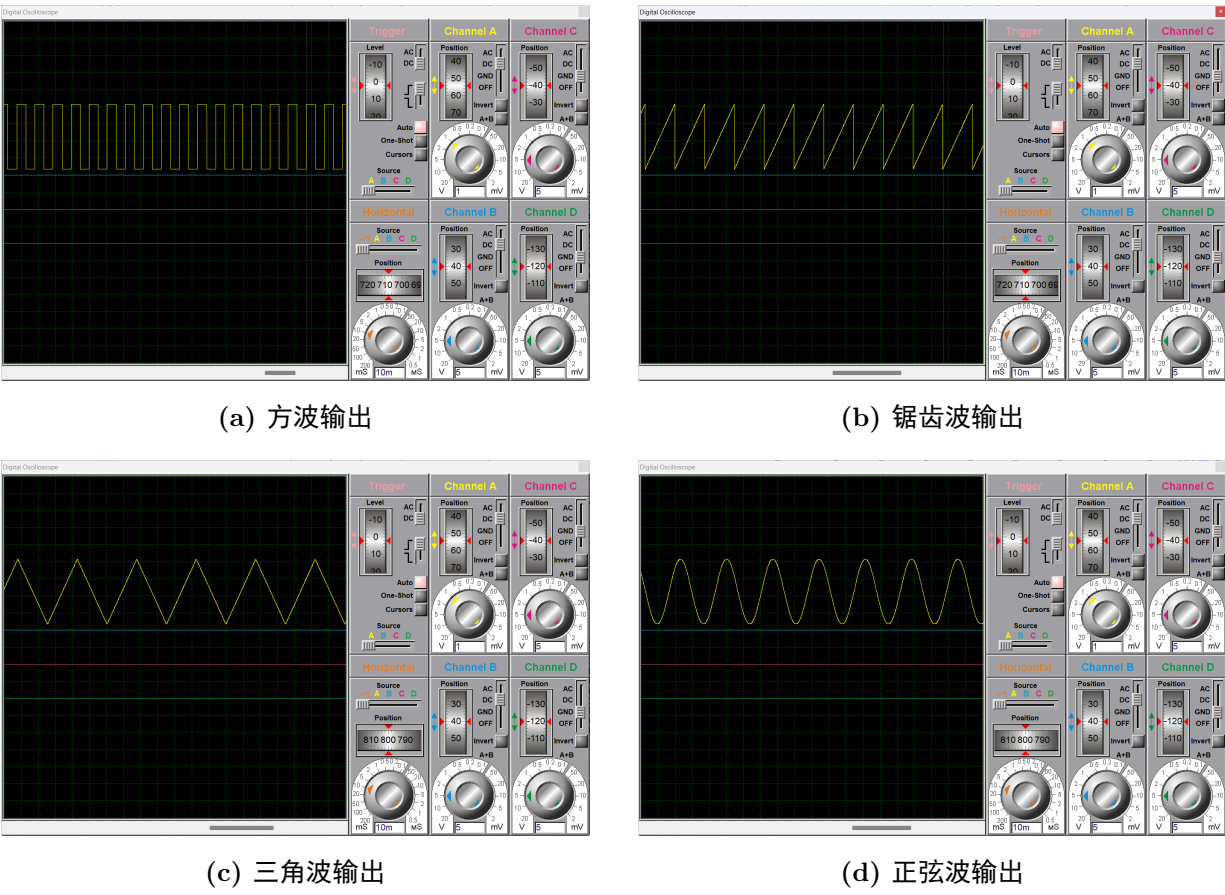


图 10 四种波形的示波器仿真结果

从示波器截图可以看出，方波在高低电平之间快速切换，锯齿波呈现线性上升后跳变回零，三角波具有近似对称的上升和下降边沿，正弦波则表现为连续平滑的周期曲线。四种波形均与程序输出规律一致。

4.3 频率调节对比

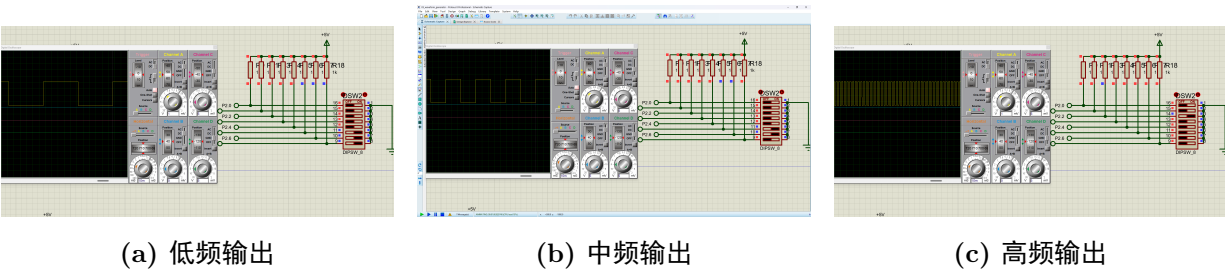


图 11 不同 P2 取值下的频率调节效果

频率对比结果表明，调节 P2 口拨码值后，同一屏幕时间范围内显示的周期数发生明显变化。低频时波形周期长、分布较稀疏；高频时周期变短、波形更密集，说明 P2 口频率控制逻辑有效。

4.4 结果验证表

表 5 波形发生器仿真结果记录

序号	测试内容	预期现象	结果
1	上电复位与默认状态	未按键时默认输出方波	符合
2	方波选择	输出高低电平交替的方波，P1.4 LED 点亮	符合
3	锯齿波选择	输出线性上升并回零的锯齿波，P1.5 LED 点亮	符合
4	三角波选择	输出线性上升再下降的三角波，P1.6 LED 点亮	符合
5	正弦波选择	输出平滑正弦曲线，P1.7 LED 点亮	符合
6	P2 频率调节	P2 值增大时输出频率升高	符合

5 问题分析与改进建议

1. 当前频率调节依赖软件延时，延时期间 CPU 不能处理其他任务，且实际频率受指令执行时间影响。若需要更稳定的频率，可使用定时器中断产生固定采样节拍。
2. DAC0832 采用直通方式，电路简单、便于实验观察，但如果后续需要与多路外设共享数据总线，可以改用锁存控制方式，提高总线时序的可控性。
3. P1 口同时承担开关输入和 LED 输出，程序需要注意写端口时保持低四位为 1。若硬件资源允许，可以将输入和指示灯分配到不同端口，降低端口复用带来的软件约束。
4. 正弦波由 92 点四分之一表生成，曲线已经较平滑。若追求更高精度，可增加表点数量；若追求更高频率，可减少表点或使用定时器配合查表输出。
5. 机械拨码开关在实际硬件中可能存在抖动。本实验在 Proteus 中影响较小，若移植到实物系统，可增加简单消抖或在周期结束时多次确认输入状态。

6 实验结论

本实验完成了基于 AT89C51、DAC0832 和 LM324 的波形发生器设计与仿真。系统通过 P0 口输出数字采样值，经 DAC0832 和运算放大器转换为模拟电压；通过 P1 口选择方

波、锯齿波、三角波和正弦波，并用 LED 指示当前波形；通过 P2 口改变软件延时，实现输出频率调节。仿真结果显示，四种波形形状清晰，频率变化效果明显，硬件连接和软件控制逻辑均满足实验要求。

实验六

空白



杭州电子科技大学

实验七 温度采集控制器实验报告

课程 微机原理与接口技术

学院 自动化学院

专业 自动化（新工科试验班）

学号 24061428

学生姓名 张跃哲

指导教师 金朝阳

完成日期 2026 年 6 月 21 日

目录

1	实验要求与实验目的	1
1.1	实验要求	1
1.2	实验目的	1
1.3	总体方案	1
2	硬件电路设计	1
2.1	单片机最小系统	2
2.2	温度采集与 ADC0808 接口	2
2.3	ADC0808 控制时序	3
2.4	数码管显示电路	4
2.5	端口分配	5
3	软件设计	5
3.1	程序设计思路	6
3.2	定时中断流程	7
3.3	温度近似换算	7
3.4	完整程序清单	7
4	仿真调试与结果分析	12
4.1	调试步骤	12
4.2	结果现象	13
4.3	问题分析	13
5	实验总结	13

1 实验要求与实验目的

1.1 实验要求

本实验要求利用 AT89C51 单片机、ADC0808 模数转换器和 AT502 热敏电阻构成温度采集控制器。系统通过热敏电阻分压电路把温度变化转换为模拟电压，再由 ADC0808 转换为 8 位数字量，最后由单片机处理后送至最左侧两位共阴极八段数码管显示。根据实验说明和开发板资源限制，本实验采用如下连接关系：

- 1. ADC0808 的 8 位数据输出口接 AT89C51 的 P1.0–P1.7。
- 2. ADC0808 的 OE 接 P2.1，START 与 ALE 共接 P2.2，EOC 接 P3.4。
- 3. ADC0808 的 CLOCK 引脚由 Proteus 数字时钟源提供 500kHz 方波。
- 4. 温度传感器采用 5kΩ NTC 热敏电阻，经过分压后接 ADC0808 的 IN0 通道。
- 5. 显示部分使用最左边两只共阴极数码管显示温度值。

1.2 实验目的

- 1. 理解 A/D 转换的基本过程，掌握模拟量采样、量化和数字量读取之间的关系。
- 2. 熟悉 ADC0808 的通道选择、启动转换、结束判断和输出允许控制方法。
- 3. 掌握 AT89C51 定时器中断、并行 I/O 口和动态数码管显示的综合应用。
- 4. 理解热敏电阻分压测温的基本原理，并能够在 Proteus 中完成电路与程序联合调试。

1.3 总体方案

系统以 AT89C51 为控制核心。热敏电阻与固定电阻构成分压网络，温度改变时热敏电阻阻值变化，ADC0808 的 IN0 输入电压随之改变。单片机每隔 1s 通过 P2.2 向 ADC0808 发出 START/ALE 脉冲，等待 P3.4 接收到转换完成信号后，打开 P2.1 对应的 OE 控制端，从 P1 口读入 8 位转换结果。程序将 ADC 值近似换算为温度值，并通过 P0 总线和两片 74HC573 锁存器驱动数码管动态显示。

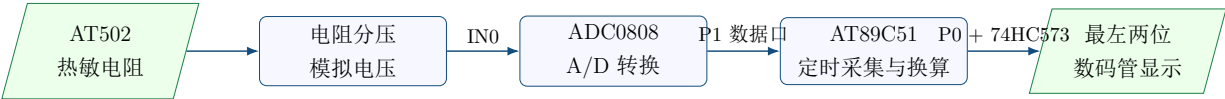
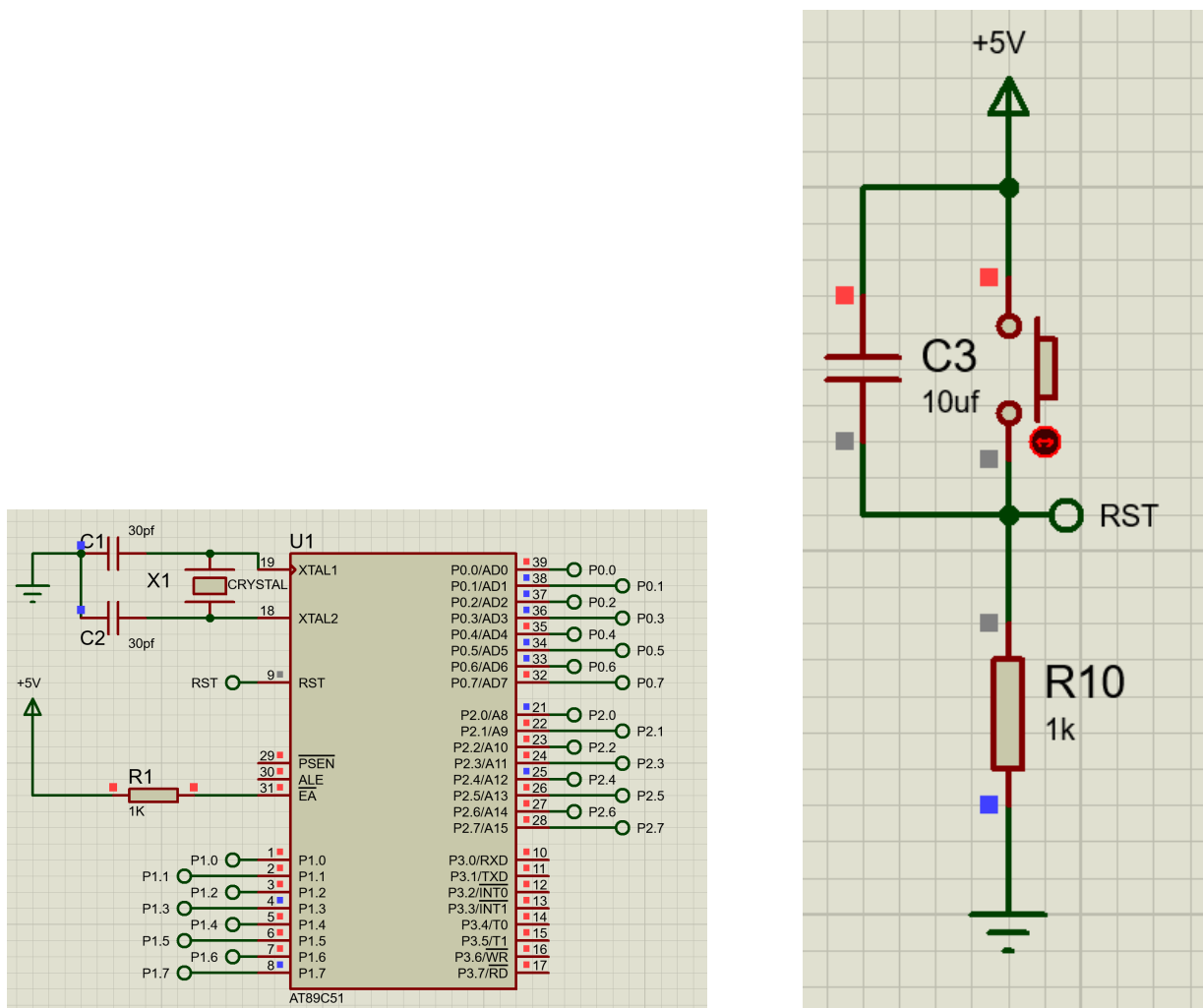


图 1 温度采集控制器总体结构

2 硬件电路设计

2.1 单片机最小系统

AT89C51 采用 +5V 供电，XTAL1 与 XTAL2 外接晶振和两个 30pF 电容构成时钟电路。EA 端通过电阻接 +5V，使单片机从片内程序存储器取指。RST 引脚连接复位网络，保证上电或按键复位时程序能够从 0000H 地址重新执行。该部分是单片机稳定运行的基础。



(a) AT89C51 供电与晶振电路

(b) 复位电路

图 2 单片机最小系统基础电路

2.2 温度采集与 ADC0808 接口

ADC0808 采用 8 位逐次逼近式 A/D 转换结构，具有 IN0-IN7 八路模拟输入。由于本实验只采集一路温度信号，因此 ADD A、ADD B、ADD C 全部接地，固定选择 IN0 通道。VREF(+) 接 +5V，VREF(-) 接 GND，因此转换结果满足近似关系：

$$D = \frac{V_{IN0}}{5V} \times 255$$

其中 D 为 P1 口读到的 8 位 ADC 数值。

温度分压采用固定上拉电阻与 NTC 热敏电阻构成。若上拉电阻为 R_3 ，热敏电阻为 R_T ，则 IN0 电压为：

$$V_{IN0} = 5V \times \frac{R_T}{R_3 + R_T}$$

NTC 热敏电阻温度升高时阻值下降，因此 IN0 电压和 ADC 数值也随温度升高而下降。为了便于仿真调试，可用电位器代替热敏电阻进行阻值变化模拟，但不宜将电位器、固定电阻和热敏电阻随意串联，否则分压关系会偏离测温模型。

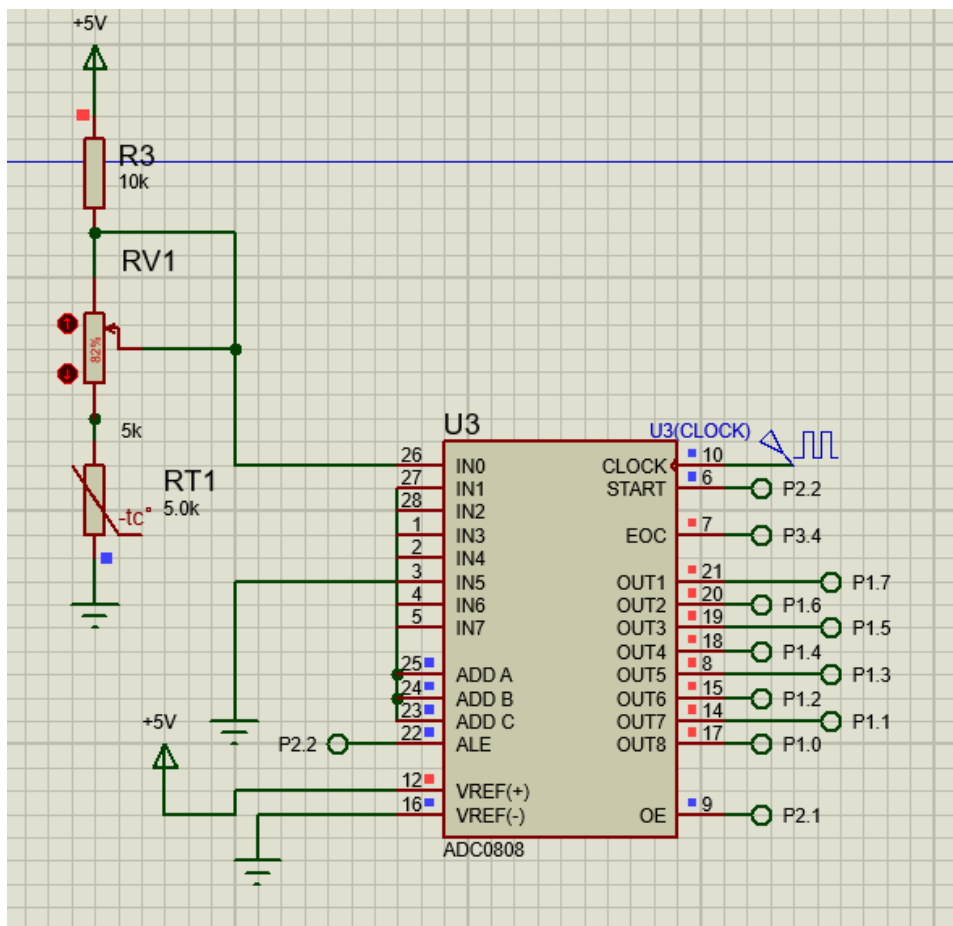


图 3 ADC0808 温度采集接口电路

2.3 ADC0808 控制时序

本实验将 START 与 ALE 共接 P2.2。采样时，单片机先使 P2.2 产生一个高电平脉冲，ADC0808 锁存通道地址并开始转换；随后程序检测 EOC 引脚状态，等待转换完成；最后置位 OE，使 ADC0808 将转换结果输出到 P1 口。读取完成后关闭 OE，避免数据总线长期被外设驱动。

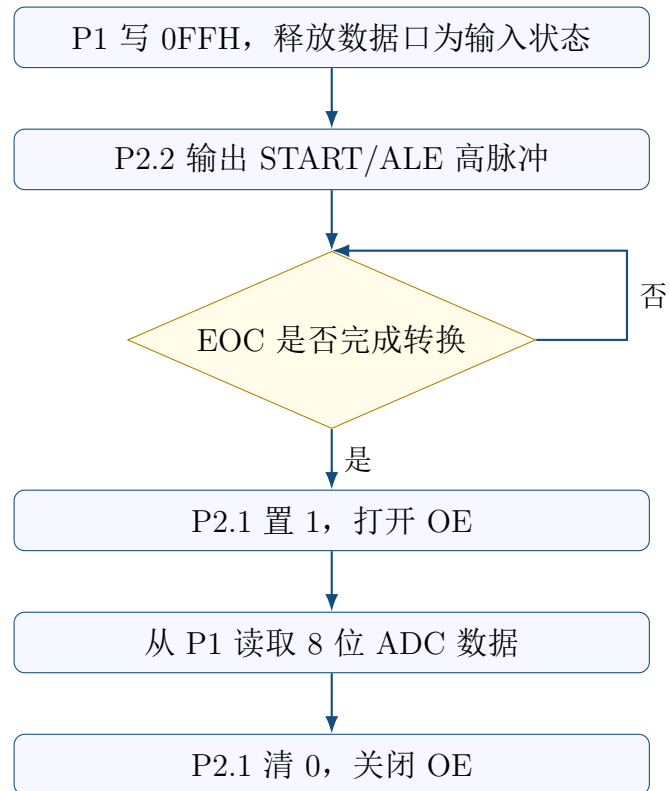


图 4 ADC0808 一次采集流程

2.4 数码管显示电路

显示部分采用一组 8 位共阴极数码管，实验只使用最左边两位。P0 作为共用数据总线，分别送入两片 74HC573：一片锁存段码，另一片锁存位选。P2.6 控制段码锁存，P2.7 控制位选锁存。程序通过快速交替刷新十位和个位，利用视觉暂留形成稳定的两位温度显示。

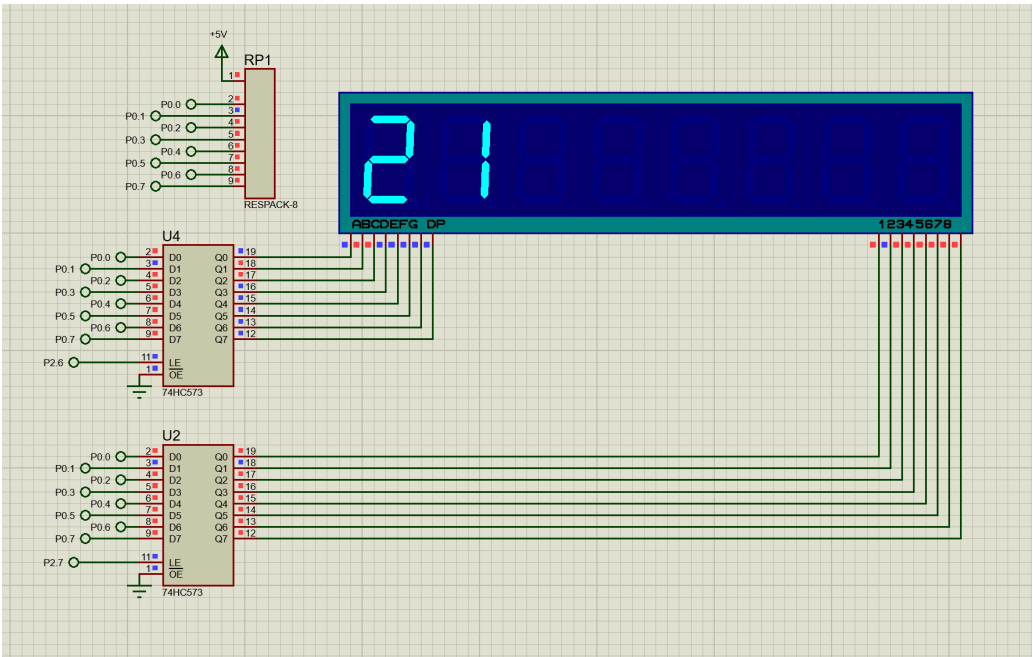


图 5 两位温度显示与 74HC573 锁存电路

2.5 端口分配

表 1 AT89C51 与外设连接关系

单片机端口	外设连接	功能说明
P1.0–P1.7	ADC0808 OUT8–OUT1	读取 8 位 A/D 转换数据，注意 OUT1 为高位
P2.1	ADC0808 OE	输出允许控制，高电平读取数据
P2.2	ADC0808 START、ALE	产生转换启动和地址锁存脉冲
P3.4	ADC0808 EOC	查询 A/D 转换结束状态
P0.0–P0.7	两片 74HC573 输入端	复用输出段码和位选码
P2.6	段码锁存 74HC573 的 LE	锁存数码管段码
P2.7	位选锁存 74HC573 的 LE	锁存数码管位选

3 软件设计

3.1 程序设计思路

程序采用“主循环刷新显示，定时中断周期采集”的结构。主循环不做复杂计算，只不断输出十位和个位的段码、位选码，保证数码管亮度稳定。定时器 0 工作在方式 1，按 50ms 进入一次中断，累计 20 次约为 1s。每满 1s 后，程序启动 ADC0808 转换、读取 P1 数据、近似换算温度，并拆分为十位和个位，供主循环显示。

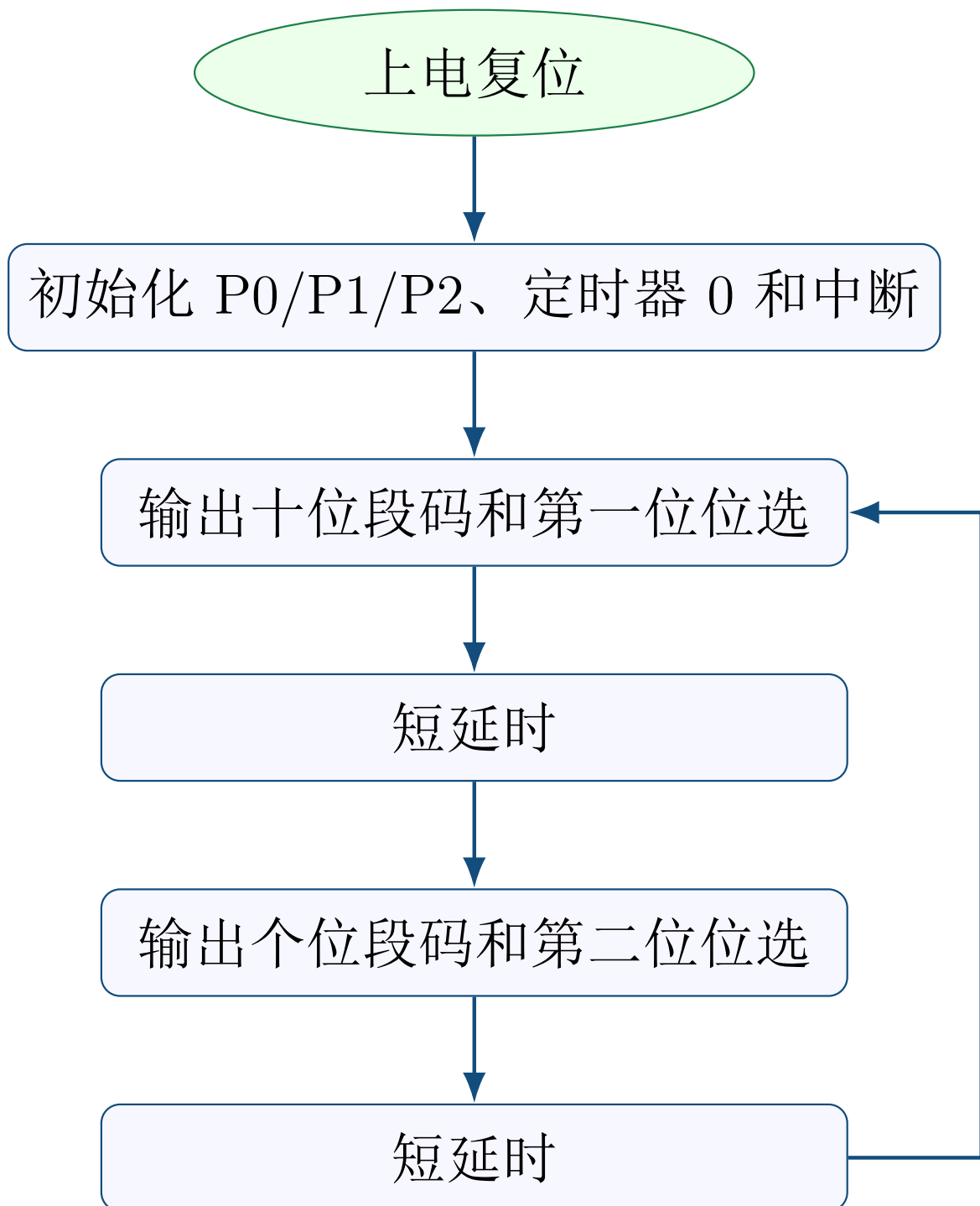


图 6 主程序显示扫描流程图

3.2 定时中断流程

定时中断既负责 1s 采集周期，又避免在主循环中使用长时间等待。中断服务程序重新装载 TH0、TL0 后递减计数变量，若未满 1s 则直接返回；若计数到 0，则重新装载计数值并执行一次 ADC 采集和温度换算。

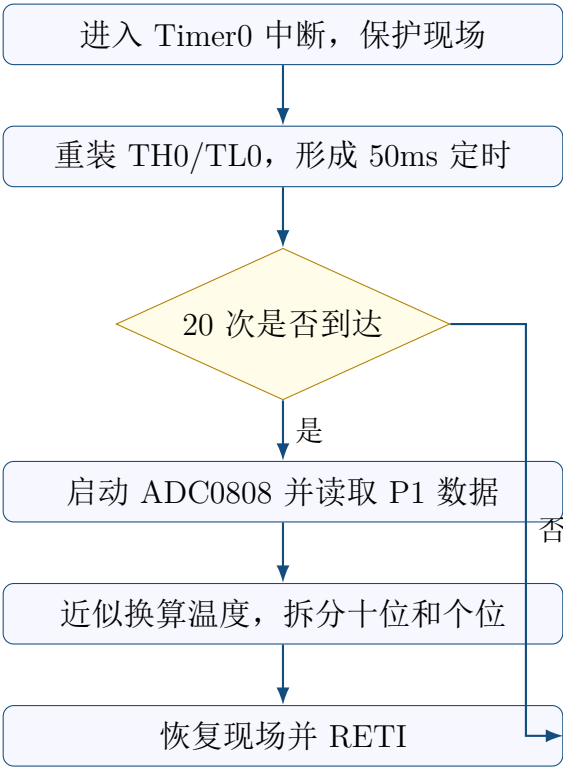


图 7 1s 周期采集的定时中断流程图

3.3 温度近似换算

本实验的重点是掌握 ADC0808 的控制和数据显示，因此程序采用分段近似方式进行温度换算。对于 5kΩ NTC 与 10kΩ 上拉电阻组成的分压电路，25℃ 附近热敏电阻约为 5kΩ，此时 ADC 数值约为：

$$D_{25} = \frac{5k}{10k + 5k} \times 255 \approx 85$$

温度升高时 ADC 值变小，温度降低时 ADC 值变大。程序以 85 为分界点，分别使用线性近似，使显示值能够随采集电压变化而改变。该方法程序短、容易在 ASEM-51 下编译，适合实验验收；若进行实际测温，应进一步使用查表法或 Steinhart-Hart 方程进行标定。

3.4 完整程序清单

Listing 1 07_temperature.A51 完整源程序

```
1 ; =====
2 ; AT89C51 + ADC0808
```

```
3 ; Proteus ASEM-51
4 ;=====
5
6 $NOMOD51
7 $INCLUDE (8051.MCU)
8
9 ADC_OE EQU 0A1H ; P2.1
10 ADC_ST EQU 0A2H ; P2.2 START + ALE
11 ADC_EOC EQU 0B4H ; P3.4
12
13 SEG_LE EQU 0A6H ; P2.6
14 BIT_LE EQU 0A7H ; P2.7
15
16 CNT EQU 30H
17 TEN EQU 31H
18 ONE EQU 32H
19
20 ORG 0000H
21 LJMP START
22
23 ORG 000BH
24 LJMP TO_INT
25
26 ORG 0100H
27
28 START:
29 MOV SP,#60H
30
31 MOV P0,#00H
32 MOV P1,#0FFH
33 MOV P2,#0FFH
34
35 CLR ADC_OE
36 CLR ADC_ST
37
38 MOV CNT,#20
39 MOV TEN,#2
40 MOV ONE,#5
41
42 MOV TMOD,#01H
```

```
43      MOV TH0,#3CH          ; 50 ms, 12 MHz
44      MOV TLO,#0B0H
45
46      SETB ETO
47      SETB EA
48      SETB TRO
49
50 MAIN:
51      ; close all digits
52      MOV PO,#0FFH
53      SETB BIT_LE
54      CLR BIT_LE
55
56      ; show ten
57      MOV DPTR,#TAB
58      MOV A,TEN
59      MOVC A,@A+DPTR
60      MOV PO,A
61      SETB SEG_LE
62      CLR SEG_LE
63
64      MOV PO,#0FEH          ; left first digit
65      SETB BIT_LE
66      CLR BIT_LE
67      LCALL DELAY
68
69      ; close all digits
70      MOV PO,#0FFH
71      SETB BIT_LE
72      CLR BIT_LE
73
74      ; show one
75      MOV DPTR,#TAB
76      MOV A,ONE
77      MOVC A,@A+DPTR
78      MOV PO,A
79      SETB SEG_LE
80      CLR SEG_LE
81
82      MOV PO,#0FDH          ; left second digit
```

```
83      SETB BIT_LE
84      CLR BIT_LE
85      LCALL DELAY
86
87      SJMP MAIN
88
89      ;=====
90      ; Timer0 interrupt: 50 ms once, read ADC every 1 s
91      ;=====
92
93 TO_INT:
94      PUSH ACC
95      PUSH B
96      PUSH PSW
97
98      MOV TH0,#3CH
99      MOV TLO,#0B0H
100
101      DJNZ CNT,TO_EXIT
102      MOV CNT,#20
103
104      ; start ADC0808
105      MOV P1,#0FFH
106      CLR ADC_OE
107
108      SETB ADC_ST
109      NOP
110      NOP
111      CLR ADC_ST
112
113 WAIT0:
114      JB ADC_EOC,WAIT0
115
116 WAIT1:
117      JNB ADC_EOC,WAIT1
118
119      SETB ADC_OE
120      NOP
121      MOV A,P1
122      CLR ADC_OE
```

```
123
124      ; A = ADC value
125      ; if ADC < 85, temperature is higher than 25
126      CJNE A,#85,CHK_ADC
127      SJMP COLD
128
129 CHK_ADC:
130      JC HOT
131
132 COLD:
133      ; temp = (160 - ADC) / 3
134      MOV B,A
135      MOV A,#160
136      CLR C
137      SUBB A,B
138      JNC COLD_OK
139      MOV A,#0
140      SJMP SPLIT
141
142 COLD_OK:
143      MOV B,#3
144      DIV AB
145      SJMP SPLIT
146
147 HOT:
148      ; temp = 25 + (85 - ADC) * 2 / 3
149      MOV B,A
150      MOV A,#85
151      CLR C
152      SUBB A,B
153      MOV B,#2
154      MUL AB
155      MOV B,#3
156      DIV AB
157      ADD A,#25
158
159 SPLIT:
160      ; split temperature to ten and one
161      MOV B,#10
162      DIV AB
```

```
163      MOV TEN,A
164      MOV ONE,B
165
166 TO_EXIT:
167      POP PSW
168      POP B
169      POP ACC
170      RETI
171
172 ;=====
173
174 DELAY:
175      MOV R6,#2
176 D1:
177      MOV R7,#250
178 D2:
179      DJNZ R7,D2
180      DJNZ R6,D1
181      RET
182
183 TAB:
184      DB 3FH,06H,5BH,4FH,66H
185      DB 6DH,7DH,07H,7FH,6FH
186
187      END
```

4 仿真调试与结果分析

4.1 调试步骤

1. 检查 AT89C51 的电源、晶振、复位和 EA 引脚连接，确认单片机可以正常运行程序。
2. 检查 ADC0808 的 VREF(+)、VREF(-)、CLOCK、START/ALE、EOC 和 OE 引脚，确认控制信号符合实验要求。
3. 检查 ADC0808 的 OUT1-OUT8 与 P1 口的连接顺序。ADC0808 的 OUT1 为高位，OUT8 为低位，若位序接反，显示值会出现大幅跳变。
4. 检查 ADD A、ADD B、ADD C 是否接地，确保固定选择 IN0 通道。
5. 加载汇编生成的 HEX 文件，启动仿真，观察最左边两位数码管是否能够显示温度

值。

6. 改变热敏电阻模型或用电位器模拟热敏电阻阻值变化，观察显示温度是否随 ADC 输入变化而更新。

4.2 结果现象

仿真运行后，最左边两位数码管能够稳定显示温度值。程序每 1s 进行一次 ADC 采样，因此改变模拟输入后，显示结果不会立即变化，而是在下一次采样周期到来后更新。显示部分采用动态扫描方式，主循环快速交替刷新十位和个位，因此肉眼观察为两位同时点亮。

4.3 问题分析

调试过程中需要特别注意两个问题。第一，ADC0808 数据位序必须与程序读取方式一致。8051 程序直接执行 `MOV A,P1`，默认 P1.7 为最高位、P1.0 为最低位；而 ADC0808 的 OUT1 为最高位、OUT8 为最低位，因此硬件连接应按高位对高位、低位对低位的方式处理。第二，用电位器模拟热敏电阻时，应保持分压模型清晰。若将固定电阻、电位器和热敏电阻全部串联，IN0 电压不再对应单一热敏电阻阻值，显示值会出现不易解释的跳变。

表 2 常见调试问题与处理方法

现象	可能原因	处理方法
数码管不亮	位选方向错误或锁存信号未连接	调整位选码，检查 P2.6/P2.7
显示值跳变很大	ADC0808 输出位序接反	将 OUT1 接 P1.7, OUT8 接 P1.0
显示一直为 00	IN0 电压超出程序换算范围或 OE 未打开	检查分压电阻和 P2.1 控制
调节电位器非线性	电位器与热敏电阻串联方式不当	用电位器单独模拟热敏电阻
采样无变化	EOC 等待逻辑或 START 脉冲异常	检查 P2.2、P3.4 和 500kHz 时钟

5 实验总结

本实验完成了基于 AT89C51 和 ADC0808 的温度采集显示系统设计。硬件方面，系统包含单片机最小系统、NTC 热敏电阻分压电路、ADC0808 转换接口和两位数码管显示电路；软件方面，程序利用定时器中断实现 1s 采集周期，并在主循环中完成数码管动态扫

描。通过本实验，进一步理解了 ADC0808 的启动、转换完成判断和数据读取过程，也认识到模拟量采集电路中分压电阻、位序连接和参考电压对结果准确性的影响。

实验中采用简化的分段线性换算实现温度显示，能够满足课堂验收中“采集、转换、显示”的功能要求。若将该系统用于更高精度的实际温度测量，还应根据热敏电阻的具体参数进行标定，并使用查表或非线性公式提高温度换算精度。